

РЕФЕРАТ

Выпускная квалификационная работа содержит 90 страниц, 37 рисунков, 9 таблиц, 13 источников, 4 приложения

ИНФОРМИРОВАНИЕ ВОДИТЕЛЕЙ, ДЕФЕКТЫ ДОРОЖНОГО ПОКРЫТИЯ, КРАУДСОРСИНГ, МИКРОСЕРВИСНАЯ АРХИТЕКТУРА, ГЕОПРОСТРАНСТВЕННЫЕ ДАННЫЕ, КОМПЬЮТЕРНОЕ ЗРЕНИЕ, YOLO, POSTGIS, FASTAPI, REACT NATIVE, МОБИЛЬНОЕ ПРИЛОЖЕНИЕ

Выпускная квалификационная работа посвящена разработке системы информирования водителей о дефектах дорожного покрытия на основе краудсорсингового подхода и технологий компьютерного зрения.

Объект исследования: процесс сбора, обработки, модерации и распространения информации о дефектах дорожного покрытия с использованием краудсорсингового подхода.

Предмет исследования: методы, алгоритмы и программные средства для реализации системы информирования участников дорожного движения о дефектах дорог, включающие серверную часть, клиентское мобильное приложение, модуль автоматической детекции на основе компьютерного зрения и механизмы визуализации данных на карте.

Цель работы: повысить уровень информированности участников дорожного движения о дефектах дорожного покрытия за счёт разработки краудсорсинговой информационной системы, обеспечивающей автоматическую детекцию дефектов на изображениях с точностью не ниже 80%, размещение подтверждённых дефектов на интерактивной карте, адресное оповещение водителей с учётом скорости и направления движения, а также поддержку ручной модерации для верификации сообщений.

Методы исследования: системный анализ предметной области; методы объектно-ориентированного программирования (Python, JavaScript); методы компьютерного зрения и машинного обучения; методы работы с геоанализом; функциональное и нагрузочное тестирование.

Результаты работы: проведён анализ существующих решений, выявлены их недостатки. Спроектирована микросервисная архитектура,

включающая сервис управления доступом (FastAPI, PostgreSQL, Redis), сервис управления дефектами (FastAPI, PostGIS, MinIO) и сервис автоматической детекции (YOLOv26n, Redis Streams). Клиентское приложение реализовано на React Native с отображением карты на базе MapLibre GL и векторных тайлов PMTiles. Реализованы алгоритмы привязки дефектов к дорожной сети, пространственных запросов, динамического оповещения водителей с учётом скорости и направления движения, а также автоматической модерации дефектов на основе модели YOLOv26n. Проведено нагрузочное тестирование: система выдержала нагрузку до 300 пользователей, среднее время ответа API составило 25,64 мс, 95-й перцентиль - 48 мс. Точность автоматической детекции (Precision) достигла 0,8707, что превышает целевой показатель 80%, производительность сервиса детекции - 30 кадров в секунду.

Выводы: разработанная система обеспечивает сбор сообщений о дефектах от пользователей, автоматическую (с точностью 87%, что превышает целевые 80%) и ручную модерацию, визуализацию на карте и адресное оповещение водителей с учётом скорости и направления движения. Цель работы достигнута, система готова к практическому использованию для информирования водителей о дефектах дорожного покрытия.

СОДЕРЖАНИЕ

Введение	7
1 Аналитический обзор методов и технологий для построения системы информирования водителей о дефектах дорожного покрытия	11
1.1 Обзор дефектов дорожного покрытия, методов их выявления и существующих решений для информирования водителей о них ...	11
1.1.1 Классификация дефектов дорожного покрытия	11
1.1.2 Влияние дефектов дорожного покрытия на безопасность движения	13
1.1.3 Методы обнаружения дефектов дорог и информирования водителей о них	13
1.1.4 Функциональные требования	16
1.1.5 Нефункциональные требования	17
1.2 Подходы, технологии и инструменты для разработки системы информирования водителей о дефектах дорог	18
1.2.1 Архитектурные подходы: монолит и микросервисы	18
1.2.2 Веб-фреймворки предназначенные для разработки серверной части	19
1.2.3 Обратные прокси и API-шлюзы	20
1.2.4 Фреймворки для разработки кроссплатформенных мобильных приложений	22
1.2.5 Системы управления базами данных, применимые для работы с геоданными	23
1.2.6 Объектные хранилища данных	25
1.2.7 In-memory системы управления базами данных	26
1.2.8 Инструменты мониторинга	27
1.2.9 Модели семейства YOLO для детекции дефектов дорожного покрытия	27
1.3 Выводы	29

2 Проектирование системы информирования водителей о дефектах дорожного покрытия	31
2.1 Архитектура системы информирования водителей о дефектах дорог	31
2.1.1 Контекст взаимодействия системы с пользователями	31
2.1.2 Компоненты системы и выбор технологий для их реализации .	32
2.1.3 Модель данных	35
2.2 Описание ключевых алгоритмов для работы с дефектами дорог . . .	38
2.2.1 Загрузка и отображение дефектов на карте	38
2.2.2 Оповещение водителя при приближении к дефекту	40
2.2.3 Создание дефекта	42
2.3 Выводы	44
3 Реализация и тестирование системы информирования водителей о дефектах дорожного покрытия	46
3.1 Реализация сервиса управления доступом	46
3.2 Реализация сервиса управления дефектами	47
3.3 Реализация сервиса автоматической детекции дефектов	48
3.4 Реализация клиентского приложения	50
3.5 Развёртывание системы информирования водителей о дефектах дорожного покрытия	52
3.6 Тестирование системы информирования водителей о дефектах дорожного покрытия	54
3.6.1 Оценка качества сервиса автоматической детекции дефектов . .	54
3.6.2 Функциональное тестирование	55
3.6.3 Нагрузочное тестирование	58
3.7 Выводы	59
Заключение	61
Список использованных источников	64
Приложение А Алгоритмы сервиса управления доступом	66

Приложение Б Алгоритмы сервиса управления дефектами	69
Приложение В Swagger-документация API	72
Приложение Г Интерфейс мобильного приложения	75

ВВЕДЕНИЕ

На сегодняшний день не существует централизованной системы оперативного информирования водителей о вновь возникших дефектах дорожного покрытия. Дорожные службы фиксируют повреждения в рамках плановых осмотров или по обращениям, но эти данные, как правило, не доводятся до участников движения. В результате водитель узнаёт о яме или выбоине чаще всего только в момент столкновения с ней. Между тем даже небольшой дефект может стать причиной потери управления автомобилем, особенно при высокой скорости, в тёмное время суток или в условиях плохой погоды.

В этой ситуации особенно ценным становится опыт самих участников движения: водители и пешеходы первыми обнаруживают новые дефекты. Если предоставить им простой и надёжный способ сообщить об угрозе, а другим участникам получить это предупреждение до попадания в зону риска, можно снизить вероятность аварий. Краудсорсинговая модель позволяет реализовать именно такой подход: превратить пользователей в распределённую сеть наблюдателей, чьи проверенные сообщения становятся основой для персонализированных и своевременных уведомлений. Такая система не требует установки дорогостоящих датчиков и не зависит от скорости бюрократических процедур, она работает за счёт активности сообщества и современных технологий.

Таким образом, разработка платформы, обеспечивающей оперативный обмен информацией о дорожных дефектах между пользователями, представляет собой не только технически реализуемое, но и социально значимое решение, направленное на снижение аварийности и повышение предсказуемости дорожной среды.

В сложившейся ситуации наблюдается противоречие между острой потребностью участников дорожного движения в оперативной и достоверной информации о дефектах дорожного покрытия и отсутствием решения, сочетающего детальную классификацию дефектов, оперативное оповещение водителей и модерацию сообщений в единой системе. Разрешение данного противоречия составляет проблему настоящего исследования.

Объектом исследования является процесс сбора, обработки, модерации и распространения информации о дефектах дорожного покрытия с использованием краудсорсингового подхода.

Предмет исследования - методы, алгоритмы и программные средства для реализации системы информирования участников дорожного движения о дефектах дорог, включающие серверную часть, клиентское мобильное приложение, модуль автоматической детекции на основе компьютерного зрения и механизмы визуализации данных на карте.

Цель работы - повысить уровень информированности участников дорожного движения о дефектах дорожного покрытия за счёт разработки краудсорсинговой информационной системы, обеспечивающей автоматическую детекцию дефектов на изображениях с точностью не ниже 80%, размещение подтверждённых дефектов на интерактивной карте, адресное оповещение водителей с учётом скорости и направления движения, а также поддержку ручной модерации для верификации сообщений.

Для достижения поставленной цели необходимо решить следующие задачи:

1. провести анализ существующих решений в области мониторинга дорожных дефектов и сформулировать требования к разрабатываемой системе;
2. спроектировать архитектуру информационной системы, включая модель взаимодействия компонентов и структуру хранения данных;
3. реализовать серверную часть системы, включающую сервисы управления доступом, управления дефектами и автоматической их детекции с применением методов компьютерного зрения;
4. реализовать мобильное приложение, обеспечивающее отображение повреждённых участков на интерактивной карте, формирование записей о выявленных дефектах с указанием координат и прикреплёнными медиафайлами, а также получение своевременных уведомлений о приближении к проблемным зонам;
5. провести тестирование разработанной системы и подготовить документацию.

В работе используются следующие методы исследования:

1. системный и сравнительный анализ для изучения предметной области и существующих аналогов;
2. методы объектно-ориентированного проектирования и программирования;
3. методы компьютерного зрения и машинного обучения для автоматической детекции объектов;
4. методы обработки и хранения пространственных данных;
5. функциональное и нагрузочное тестирование программных систем.

Практическая значимость работы состоит в реализации серверной части и клиентского мобильного приложения. Разработанная система может быть использована для оперативного информирования водителей о дефектах дорог, а также внедрена в качестве платформы для взаимодействия граждан с дорожными службами и органами контроля.

В результате выполнения работы будет разработан рабочий прототип информационной системы, основанной на краудсорсинге, который обеспечивает:

1. сбор сообщений от пользователей о вновь выявленных дефектах дорожного покрытия с привязкой к географическим координатам;
2. автоматическое распознавание дефектов дорожного покрытия на загружаемых пользователем изображениях с использованием модели компьютерного зрения;
3. модерацию поступающих данных для исключения дубликатов и ложных сообщений;
4. отображение подтверждённых дефектов на интерактивной карте в мобильном приложении;
5. оповещение пользователей, находящихся в непосредственной близости от зарегистрированного дефекта.

Система будет реализована в виде мобильного приложения с адаптивным интерфейсом и не будет зависеть от внешних дорогостоящих компонентов.

Текст выпускной квалификационной работы состоит из введения, трёх глав, заключения, списка использованных источников и приложений. Первая глава посвящена анализу существующих решений в области мониторинга дорожных дефектов, сравнительному анализу аналогов и определению требований к системе. Вторая глава содержит проектирование архитектуры системы, модели данных и ключевых алгоритмов. Третья глава описывает практическую реализацию всех компонентов системы, вопросы развёртывания и результаты тестирования. В заключении подводятся итоги работы и определяются направления дальнейшего развития.

1 Аналитический обзор методов и технологий для построения системы информирования водителей о дефектах дорожного покрытия

1.1 Обзор дефектов дорожного покрытия, методов их выявления и существующих решений для информирования водителей о них

1.1.1 Классификация дефектов дорожного покрытия

Для корректного проектирования системы информирования водителей о дефектах дорожного покрытия необходимо опираться на нормативную базу, регламентирующую эксплуатационное состояние автомобильных дорог. Основным действующим документом в Российской Федерации является ГОСТ Р 50597-2017 «Дороги автомобильные и улицы. Требования к эксплуатационному состоянию, допустимому по условиям обеспечения безопасности дорожного движения. Методы контроля» [2].

Согласно данному стандарту, **дефект** определяется как несоответствие транспортно-эксплуатационных показателей конструктивных элементов дорог и улиц, дорожных сооружений и элементов обустройства требованиям настоящего стандарта.

В рамках исследования рассматриваются следующие виды дефектов дорожного покрытия, наиболее часто встречающиеся на российских дорогах и представляющие опасность для участников движения.

Трещины являются наиболее распространёнными деформациями покрытий. В зависимости от ориентации и характера распространения различают:

- **Продольная трещина** - трещина, идущая параллельно оси дороги. Образуется вследствие усталости материала или температурных деформаций.
- **Поперечная трещина** - трещина, пересекающая ось дороги. Возникает из-за резких перепадов температур или циклических нагрузок.

- **Сетка трещин** - множественные взаимосвязанные трещины, образующие замкнутые ячейки. Свидетельствует об усталостном разрушении дорожной одежды.
- **Заделанная трещина** - трещина, прошедшая ремонт. Требуется контроля качества выполненного ремонта.

Локальные разрушения покрытия

- **Выбоина** - местное разрушение дорожного покрытия, имеющее вид углубления с резко очерченными краями. Согласно приложению А ГОСТ Р 50597-2017, является наиболее опасным видом дефекта.
- **Колейность** - деформация покрытия с образованием продольных углублений по полосам наката. Возникает из-за остаточных деформаций в покрытии или основании.

Дефекты разметки и элементов обустройства

- **Смытая разметка пешеходного перехода** - нечёткая или частично отсутствующая горизонтальная разметка типа «зебра».
- **Смытая разметка полос** - нечёткая или стёртая разметка, разделяющая полосы движения.
- **Люк колодца** - выступающий или проваливающийся относительно уровня покрытия люк инженерных коммуникаций.

Ремонтные вмешательства

- **Заплата** - участок покрытия, отремонтированный методом карты. Часто имеет неровности на стыках с существующим покрытием.

Представленная классификация охватывает основные виды дефектов, с которыми сталкиваются участники дорожного движения. Понимание особенностей каждого типа дефекта необходимо для определения требований к системе сбора и отображения информации, а также для разработки сценариев оповещения водителей в зависимости от типа обнаруженного дефекта.

1.1.2 Влияние дефектов дорожного покрытия на безопасность движения

В статье «Влияние состояния дорожного полотна и дорожных условий на безопасность дорожного движения» авторы отмечают, что одной из главных причин дорожно-транспортных происшествий на участках с неудовлетворительным состоянием дорожного покрытия является столкновение автомобилей при резком торможении перед неровностью, а также столкновения при попытке объезда неровностей с выездом на полосу встречного движения [8].

По данным авторов, за 2021 год зарегистрировано почти 53 тысячи ДТП из-за неудовлетворительного состояния дорог, в которых погибло 25 тысяч и ранено 68 тысяч человек. Неровность покрытия, по данным ГИБДД, является причиной 14% ДТП, связанных с неблагоприятными дорожными условиями.

Неровности дорожного полотна вызывают колебания трансмиссии автомобиля, что может привести к потере управляемости. Колебания прицепов и полуприцепов увеличивают динамический коридор движения и вероятность столкновения. Кроме того, наличие неровностей повышает утомляемость водителей, отвлекает их внимание от дороги и снижает пропускную способность.

Значительное число ДТП в тёмное время суток объясняется резким ухудшением условий зрительного восприятия, а также возможным ослеплением водителей отражённым светом фар от воды, заполняющей неровности.

Таким образом, состояние дорожного полотна напрямую влияет на безопасность движения, что обосновывает необходимость создания системы оперативного информирования водителей о дефектах покрытия.

1.1.3 Методы обнаружения дефектов дорог и информирования водителей о них

Обнаружение дефектов дорожного покрытия и информирование водителей о них традиционно осуществляется следующими методами.

Выявление дефектов производится с помощью:

- **Плановых осмотров и передвижных лабораторий** - дорожные службы проводят регулярные объезды закреплённых участков, фиксируя обнаруженные дефекты в актах. Для более точной диагностики используются специализированные автомобили-диагносты с измерительным оборудованием (профилометры, лазерные сканеры). Недостатками являются низкая периодичность, высокие затраты, а также отсутствие передачи информации о выявленных дефектах участникам дорожного движения. В результате водитель узнаёт о дефекте только при непосредственном столкновении с ним.
- **Обращений граждан** - заявки от водителей и пешеходов о замеченных дефектах через государственные порталы и приложения. Однако обработка заявок занимает длительное время, информация о подтверждённых дефектах не передаётся другим водителям, а сами данные используются преимущественно для планирования ремонтных работ, а не для информирования участников движения.

Информирование водителей об уже выявленных дефектах осуществляется с помощью:

- **Дорожных знаков** - временные знаки «Неровная дорога» (1.16) устанавливаются на участках с известными дефектами, но не содержат информации о точном местоположении и степени опасности каждого дефекта.
- **Навигационных приложений** - Waze позволяет водителям отмечать выбоины как отдельный тип дорожной опасности. Яндекс.Карты и 2ГИС предоставляют возможность отмечать «дорожные работы», «ДТП» и «камеры», но не поддерживают классификацию конкретных дефектов покрытия (трещины, колейность, смытая разметка и т.п.). Кроме того, данные отметки, как правило, не проходят модерацию и не интегрированы в единую базу дефектов.

Для более детального анализа рассмотрим существующие программные решения, предоставляющие информацию о состоянии дорожного покрытия и дорожных событиях.

Яндекс.Карты позволяют пользователям отмечать «дорожные работы», «ДТП» и «камеры». Возможности для отметки конкретных дефектов (выбоины, трещины, колейность, дефекты разметки) отсутствуют. Информация о состоянии покрытия передаётся косвенно - через отметки о ремонтных работах.

2ГИС предоставляет возможность отмечать «дорожные работы» и «ДТП», но не поддерживает классификацию дефектов дорожного покрытия. Информация о выбоинах, трещинах или смытой разметке отсутствует.

Waze позволяет водителям отмечать выбоины и другие дорожные опасности. Сообщения проходят модерацию и отображаются на карте для других водителей в реальном времени. Однако Waze не покрывает все типы дефектов, выделенные в настоящем исследовании (трещины, колейность, дефекты разметки).

Проект «Дорожная инспекция / Карта убитых дорог» позволяет фиксировать отдельные дефекты: ямы, провалившиеся люки, стёршуюся разметку, неработающие светофоры, сломанные знаки. Заявки проходят обязательную модерацию перед добавлением в общую базу. Однако проект ориентирован на взаимодействие с государственными органами, а не на оперативное информирование водителей. Отсутствует система оповещений при приближении к дефекту.

Сравнительный анализ рассмотренных методов и решений позволяет сделать следующие выводы:

- Традиционные методы (плановые осмотры, дорожные знаки) характеризуются низкой оперативностью и отсутствием детальной информации о дефектах.
- Яндекс.Карты и 2ГИС не поддерживают детальную классификацию дефектов дорожного покрытия.
- Waze выделяется наличием прямой отметки опасностей, но не покрывает все типы дефектов, рассматриваемые в работе.
- «Карта убитых дорог» предоставляет детальную классификацию дефектов с ручной модерацией, но ориентирована на взаимодействие с государственными органами, а не на оповещение водителей.

Таким образом, существующие подходы и решения не обеспечивают одновременно:

- детальную классификацию различных типов дефектов;
- оперативное оповещение водителей;
- модерацию поступающих сообщений.

Выявленные недостатки подтверждают актуальность разработки системы, сочетающей все перечисленные функции.

1.1.4 Функциональные требования

Подсистема управления доступом: система должна обеспечивать регистрацию и аутентификацию пользователей, управление сессиями (вход, выход, обновление токенов, выход из всех устройств), просмотр и редактирование профиля, управление ролями и разрешениями (создание, чтение, обновление, удаление, назначение и отзыв), а также проверку прав доступа при выполнении операций.

Подсистема управления дефектами: система должна обеспечивать создание дефекта с указанием типа, координат, фотографий и описания; привязку координат к дорожной сети; просмотр списка дефектов с фильтрацией по статусу и типу; просмотр детальной информации; редактирование и удаление собственных дефектов пользователем; модерацию дефектов (подтверждение или отклонение с указанием причины); автоматическое изменение статуса; поиск подтверждённых дефектов в заданном радиусе и внутри прямоугольной области для отображения на карте; фильтрацию результатов поиска по типам дефектов и уровню опасности.

Подсистема автоматической детекции дефектов: система должна обеспечивать автоматическую модерацию дефектов, находящихся в статусе «на проверке». Сервис получает изображения, привязанные к дефекту, и выполняет их анализ с использованием модели компьютерного зрения. Точность автоматической детекции (Precision) должна составлять не менее 80%. Время от подачи сообщения до автоматического подтверждения дефекта не должно превышать 3 минут. В случае, если на изображениях обнаружен дефект, соответствующий заявленному типу (или если уверенность модели превышает заданный порог), сервис автоматически

переводит дефект в статус «подтверждён». Если ожидаемый дефект не обнаружен или уверенность модели низкая, дефект остаётся в статусе «на проверке» для дальнейшей ручной проверки модератором. Сервис должен обеспечивать обработку изображений (изменение размера, улучшение контраста, фильтрацию) перед передачей в модель.

Клиентское приложение: система должна обеспечивать отображение подтверждённых дефектов на интерактивной карте, фильтрацию дефектов по типу и уровню опасности, создание новых дефектов с привязкой геопозиции и добавлением фотографий, просмотр и редактирование собственных дефектов с отслеживанием их статуса, а также автоматическое получение уведомлений при приближении к подтверждённому дефекту.

Подсистема мониторинга: система должна обеспечивать сбор метрик о работе сервисов (время ответа, количество запросов, частота ошибок, размер очереди сообщений, загрузка CPU и памяти), визуализацию собираемых метрик для анализа состояния системы, а также оповещение администратора при возникновении критических ситуаций.

1.1.5 Нефункциональные требования

Производительность: время ответа API на запросы получения дефектов не должно превышать 1–2 секунд при нагрузке до 300 одновременно работающих пользователей; время отклика приложения при отображении карты должно обеспечивать плавное масштабирование без заметных задержек.

Безопасность: система должна обеспечивать защиту персональных данных пользователей; пароли должны храниться в хэшированном виде; аутентификация должна осуществляться с использованием токенов с ограниченным сроком действия; доступ к операциям должен быть разграничен в зависимости от роли пользователя.

Доступность: система должна сохранять работоспособность при отказе отдельных сервисов; допустимое время недоступности сервера — не более 0,1% времени в месяц.

Удобство использования: интерфейс приложения должен быть интуитивно понятным и не требовать специального обучения; приложение

должно быть адаптировано для использования на мобильных устройствах с различными размерами экрана; процесс создания дефекта должен занимать минимальное время (не более 3–4 действий).

Масштабируемость: горизонтальное масштабирование сервисов должно достигаться добавлением дополнительных экземпляров без изменения исходного кода; архитектура должна позволять масштабировать компоненты независимо друг от друга в зависимости от нагрузки.

Надёжность: система должна корректно обрабатывать ошибки ввода пользователя и сетевые сбои; при отсутствии сетевого соединения клиентское приложение должно сохранять введённые данные для последующей отправки после восстановления связи.

1.2 Подходы, технологии и инструменты для разработки системы информирования водителей о дефектах дорог

Для обоснования выбора технологического стека разрабатываемой системы необходимо провести анализ существующих архитектурных подходов, веб-фреймворков, систем управления базами данных и инструментов мониторинга, которые могут быть использованы при реализации проекта.

1.2.1 Архитектурные подходы: монолит и микросервисы

При проектировании программных систем выделяют два основных архитектурных подхода: монолитная архитектура и микросервисная архитектура. Подробное описание этих подходов и принципов их применения представлено в работах Ричардсона и Ньюмена [5, 4].

Монолитная архитектура представляет собой единое приложение, включающее все компоненты системы. Все модули работают в рамках одного процесса и используют общую базу данных. Преимуществами монолитного подхода являются простота разработки на начальном этапе, отсутствие дополнительных затрат на межсервисное взаимодействие и упрощённое развёртывание. Недостатки: сложность масштабирования (только всего приложения целиком), низкая гибкость (изменение одного модуля требует пересборки всего приложения), высокая связанность компонентов.

Микросервисная архитектура предполагает разделение системы на несколько независимых сервисов, каждый из которых отвечает за свою функциональную область. Сервисы взаимодействуют через сеть и могут развёртываться независимо. Преимущества: независимое масштабирование, независимость разработки и развёртывания, устойчивость к отказам. Недостатки: повышенная сложность организации взаимодействия между сервисами, дополнительные затраты на сетевые вызовы.

В статье «Сравнительный анализ архитектурных подходов к разработке REST API для высоконагруженных систем» отмечается, что выбор между монолитной и микросервисной архитектурой зависит от требований к масштабируемости и нагрузке системы [7]. Для высоконагруженных систем с необходимостью независимого масштабирования компонентов предпочтительнее микросервисный подход. Результаты нагрузочного тестирования показали, что монолитная архитектура демонстрирует лучшие показатели задержки в пределах одного сервера, в то время как микросервисная архитектура выигрывает в распределённых сценариях и при необходимости горизонтального масштабирования.

Характеристики рассматриваемых подходов приведены в таблице 1.

Таблица 1 — Сравнение архитектурных подходов

Критерий	Монолитная	Микросервисная
Масштабирование	Только целиком	Независимое
Развёртывание	Единое	Независимое
Связанность компонентов	Высокая	Низкая
Устойчивость к отказам	Низкая	Высокая
Технологическая гибкость	Отсутствует	Присутствует
Сложность разработки	Низкая	Высокая

1.2.2 Веб-фреймворки предназначенные для разработки серверной части

Для реализации серверной части рассматриваются следующие варианты: FastAPI (Python), Django REST Framework (Python), ASP.NET Core (C#), NestJS (TypeScript), Spring Boot (Java).

В статье «Сравнительный анализ архитектурных подходов к разработке REST API для высоконагруженных систем» проведено нагрузочное тестирование перечисленных фреймворков [7]. В ходе тестирования измерялись средняя задержка, пропускная способность, p95 задержка и процент ошибок. Результаты представлены в таблице 2.

Таблица 2 — Результаты нагрузочного тестирования веб-фреймворков

Фреймворк	Средняя задержка, мс	Пропускная способность, req/s	p95 задержка, мс	Ошибки, %
FastAPI	43	1900	87	0,2
ASP.NET Core	52	1740	102	0,4
NestJS	61	1320	123	0,5
Django REST	128	730	246	1,1
Spring Boot	97	960	205	0,9

Как показано в таблице, наилучшие результаты по всем показателям демонстрируют FastAPI и ASP.NET Core. Эти фреймворки обеспечивают минимальную задержку, высокую пропускную способность и низкий процент ошибок, что делает их предпочтительными для реализации высоконагруженных систем.

1.2.3 Обратные прокси и API-шлюзы

В микросервисной архитектуре для маршрутизации запросов, балансировки нагрузки и ограничения частоты запросов используются обратные прокси-серверы или API-шлюзы.

Рассматриваются следующие решения:

1. **NGINX** - высокопроизводительный веб-сервер и обратный прокси с поддержкой балансировки нагрузки (round-robin, least connections, IP hash), ограничения частоты запросов (rate limiting), TLS-терминации и кэширования. Обладает низким потреблением ресурсов и высокой пропускной способностью. Имеет как коммерческую (NGINX Plus), так и open-source версию.
2. **HAProxy** - специализированный программный балансировщик нагрузки и прокси-сервер с широкими возможностями

маршрутизации и мониторинга состояния серверов (health checks). Поддерживает множество алгоритмов балансировки и используется в высоконагруженных системах.

3. **Traefik** - современный обратный прокси с интеграцией в экосистему контейнеризации (Docker, Kubernetes). Автоматически обнаруживает сервисы и обновляет конфигурацию маршрутизации. Поддерживает автоматическое получение TLS-сертификатов через Let's Encrypt.
4. **Envoy** - высокопроизводительный прокси-сервер, разработанный для облачных и микросервисных архитектур. Используется в качестве sidecar-прокси в сервисных сетках (service mesh) на базе Istio.

Сравнение обратных прокси и API-шлюзов представлено в таблице 3.

Таблица 3 — Сравнение обратных прокси и API-шлюзов

Критерий	NGINX	HAProxy	Traefik	Envoy
Балансировка нагрузки	Да	Да	Да	Да
Rate limiting	Да	Да	Да	Да
TLS-терминация	Да	Да	Да	Да
Интеграция с Docker	Ограниченная	Нет	Встроенная	Частичная
Автоматическое обнаружение сервисов	Нет	Нет	Да	Да
Простота настройки	Средняя	Средняя	Высокая	Низкая
Open Source	Да (основная версия)	Да	Да	Да

На основе проведённого анализа можно сделать следующие выводы:

1. **NGINX** - наиболее распространённое и проверенное решение, обеспечивающее высокую производительность и низкое потребление ресурсов. Подходит для большинства задач маршрутизации и балансировки нагрузки.

2. **Traefik** - удобный выбор для сред с активным использованием контейнеров, благодаря встроенной интеграции с Docker и автоматической настройке маршрутов.
3. **HAProxy** - специализированное решение для высоконагруженных систем с требованиями к отказоустойчивости.
4. **Envoy** - используется преимущественно в составе service mesh для микросервисных архитектур.

1.2.4 Фреймворки для разработки кроссплатформенных мобильных приложений

Для реализации клиентского мобильного приложения необходимо выбрать инструмент разработки. В статье К.Р. Адещенко проведён сравнительный анализ ключевых кроссплатформенных фреймворков для разработки мобильных приложений: React Native, Flutter, Xamarin, Ionic и Kotlin Multiplatform [1].

Данные фреймворки позволяют создавать мобильные приложения для нескольких платформ (Android, iOS, Windows) на основе единой кодовой базы и обеспечивают создание интерфейса, максимально приближенного к нативным приложениям.

В рамках сравнительного анализа автором были рассмотрены следующие критерии: производительность, визуальная составляющая, доступ к нативным функциям, сложность освоения и размер приложений.

React Native демонстрирует высокую производительность, обусловленную нативной отрисовкой пользовательского интерфейса и прямым взаимодействием с платформенными API. Визуализация интерфейса близка к нативной, доступ к функциям устройства реализован через специальные обёртки.

Flutter обеспечивает выдающуюся производительность, не уступающую нативным приложениям, за счёт собственного механизма визуализации Flutter Engine. Визуальная составляющая полностью идентична между операционными системами.

Xamarin позволяет достичь высокой скорости работы за счёт связи с нативными API через обёртки C#. Пользовательский интерфейс максимально приближен к платформенному.

Kotlin Multiplatform ориентирован на мобильные платформы и позволяет создавать приложения для Android и iOS с использованием общей кодовой базы на языке Kotlin. Бизнес-логика выполняется нативно, что обеспечивает высокую производительность. Взаимодействие общей кодовой базы с платформенно-зависимой частью осуществляется через специальный интерфейс, позволяющий гибко наращивать функциональность под особенности конкретных ОС.

Ionic Framework базируется на веб-технологиях (HTML, CSS, JavaScript). Интерпретация кода накладывает ограничения на производительность - приложения на Ionic уступают по скорости работы как нативным решениям, так и другим кроссплатформенным фреймворкам.

Таким образом, проведённый анализ демонстрирует, что React Native, Flutter, Xamarin и Kotlin Multiplatform обеспечивают высокую производительность и хороший доступ к нативным функциям, что делает их предпочтительными для создания приложений с высокими требованиями к скорости работы. Ionic, основанный на веб-технологиях, уступает им по производительности.

1.2.5 Системы управления базами данных, применимые для работы с геоданными

Для хранения геопространственных данных рассматриваются следующие варианты: PostgreSQL с расширением PostGIS, Microsoft SQL Server (с модулем Spatial), Oracle Spatial and Graph, MongoDB.

В статье «Сравнительный анализ систем управления базами данных при реализации распределенного хранилища геоданных» отмечается, что ключевыми критериями выбора СУБД для геоданных являются поддержка геопространственных типов, наличие пространственных индексов, производительность выполнения геозапросов и масштабируемость в распределённой среде [6].

Результаты проведённого сравнительного анализа представлены в таблице 4.

Таблица 4 – Сравнение систем управления базами данных

Критерий	PostgreSQL/ PostGIS	Microsoft SQL Server	Oracle Spatial	MongoDB
Поддержка стандартов OGC	Полная	Базовая	Полная	Частичная
Геопространственные типы	Векторные, растры, 3D	Векторные	Векторные, растры, 3D	Растры, метаданные
Пространственные индексы	Да (GiST)	Да	Да (R-tree)	Ограниченно
Поддержка ACID	Да	Да	Да	Нет
Поддержка распределённых БД	Через Citus	Always On	Oracle RAC	Шардирование

На основе проведённого анализа можно сделать следующие выводы:

1. **PostgreSQL/PostGIS** является наиболее универсальной системой, обеспечивающей сбалансированное сочетание функциональности, открытости и масштабируемости. Поддерживает все типы геоданных (векторные, растровые, облака точек) и стандарты OGC.
2. **Oracle Spatial and Graph** - оптимальный выбор для корпоративных и государственных задач, где важны масштабируемость, отказоустойчивость и сложные топологические операции.
3. **Microsoft SQL Server** демонстрирует надёжную работу с векторными данными, хорошую масштабируемость и интеграцию с Windows-средой.
4. **MongoDB** эффективна для хранения метаданных и растровых файлов в микросервисных архитектурах, но не подходит для сложных пространственных запросов.

1.2.6 Объектные хранилища данных

Для хранения файлов, не являющихся структурированными данными, в современных распределённых системах используются объектные хранилища. В разрабатываемой системе требуется хранить фотографии дефектов, загружаемые пользователями при создании сообщения о дефекте.

Рассматриваются следующие объектные хранилища:

1. **Amazon S3 (Amazon Simple Storage Service)** - проприетарное облачное решение, являющееся стандартом де-факто для объектного хранения [9]. Предоставляет высокую доступность, масштабируемость и простой API для загрузки и скачивания файлов.
2. **MinIO** - открытая реализация S3-совместимого объектного хранилища, предназначенная для развёртывания в собственной инфраструктуре [12]. Полностью совместима с S3 API, что позволяет использовать стандартные клиентские библиотеки.
3. **Ceph** - распределённая файловая система с поддержкой объектного хранения, обеспечивающая высокую отказоустойчивость и масштабируемость [10]. Требуется более сложная настройка по сравнению с MinIO.

Сравнение объектных хранилищ представлено в таблице 5.

Таблица 5 – Сравнение объектных хранилищ

Критерий	Amazon S3	MinIO	Ceph
Лицензия	Проприетарная	Open Source (AGPLv3)	Open Source (LGPLv2)
Развёртывание	Облачное	Self-hosted	Self-hosted
Совместимость с S3 API	Нативная	Полная	Частичная (RADOS Gateway)
Сложность настройки	Низкая	Низкая	Высокая
Подходит для self-hosted	Нет	Да	Да

1.2.7 In-memory системы управления базами данных

В обзоре рынка in-memory систем управления базами данных отмечается, что такие системы хранят информацию непосредственно в оперативной памяти, что обеспечивает высокую скорость доступа и делает их идеальными для задач кэширования и управления сессиями [3]. Основные недостатки - ограниченный объём хранимых данных и энергозависимость.

Среди наиболее популярных in-memory решений можно выделить следующие:

1. **Redis** - высокопроизводительная система с открытым исходным кодом. Поддерживает различные структуры данных (строки, хеши, списки, множества), настройку времени жизни (TTL) для записей, а также сохранение данных на диск. На сегодняшний день Redis является наиболее распространённым решением для кэширования и хранения сессионных данных.
2. **Memcached** - высокопроизводительная многопоточная система кэширования. Поддерживает только базовые операции ключ-значение и не имеет механизма сохранения данных на диск.
3. **Tarantool** - СУБД с открытым исходным кодом, разработанная компанией VK. Позволяет хранить различные типы информации (таблицы, документы), имеет индексы, инструменты для шардирования, а также поддерживает сохранение данных на диск.

Сравнение основных характеристик in-memory систем представлено в таблице 6.

Таблица 6 — Сравнение in-memory систем управления базами данных

Характеристика	Memcached	Redis	Tarantool
Компания разработчика	Danga Interactive	Redis Labs	VK
Лицензия	Open Source (BSD)	Open Source (Redis Source Available License v2)	Open Source (BSD)

Продолжение таблицы 6 — Сравнение in-memory систем управления базами данных

Характеристика	Memcached	Redis	Tarantool
Открытый исходный код	Да	Да	Да
Тип базы данных	Ключ-значение	Ключ-значение	Мультидомальная, ключ-значение
Режимы хранения	Оперативная память	Оперативная память	Оперативная память
Транзакции	Нет	Да	Да

1.2.8 Инструменты мониторинга

Для организации мониторинга системы рассматриваются следующие инструменты, широко применяемые в микросервисных архитектурах:

1. **Сбор метрик:** Prometheus - система с pull-моделью сбора метрик, которая регулярно опрашивает HTTP-эндпоинты сервисов. Для микросервисной архитектуры предпочтительнее pull-модель, так как она не требует настройки передачи данных из каждого экземпляра сервиса [13].
2. **Визуализация:** Grafana - наиболее распространённый инструмент для построения дашбордов, поддерживающий множество источников данных, включая Prometheus [11].
3. **Оповещение:** Alertmanager - встроенный компонент экосистемы Prometheus, позволяющий настраивать отправку уведомлений через Telegram, email, Webhook и другие каналы [13].

1.2.9 Модели семейства YOLO для детекции дефектов дорожного покрытия

Для автоматического обнаружения дефектов дорожного покрытия на изображениях, загружаемых пользователями, в составе разрабатываемой системы необходим модуль компьютерного зрения. В данном разделе проводится анализ современных моделей детекции объектов и экспериментальное сравнение архитектур семейства YOLO на специализированном наборе данных дорожных дефектов.

Свёрточные нейронные сети являются основой большинства современных систем детекции объектов. Среди одностадийных детекторов семейство YOLO (You Only Look Once) обеспечивает наилучшее соотношение скорости и точности. Основные эволюционные версии включают YOLOv8, YOLOv11, YOLOv12 и YOLOv26, каждая из которых вносит улучшения в архитектуру.

В рамках работы проведено экспериментальное сравнение четырёх моделей семейства YOLO на задаче детекции дефектов дорожного покрытия: YOLOv8n, YOLOv11n, YOLOv12n и YOLOv26n.

Обучение проводилось на наборе данных RDD 2022, содержащем размеченные изображения дорожных дефектов. Набор данных включает следующие классы: продольная трещина, поперечная трещина, сетка трещин, иные повреждения (заплаты, колеиность и др.) и выбоина.

Перед подачей в модель изображения проходили последовательность операций предобработки, направленных на улучшение локального контраста и выделение границ дефектов: изменение размера изображения до 640×640 пикселей с билинейной интерполяцией, применение CLAHE для улучшения локального контраста и двусторонняя фильтрация для удаления шума с сохранением границ дефектов. На рисунке 1 представлено сравнение исходного изображения и изображения после обработки.



Рисунок 1 — Сравнение исходного изображения (слева) и изображения после обработки (справа)

Все модели обучались в течение 100 эпох на одинаковых настройках. Основные гиперпараметры: размер изображения - 640×640 пикселей,

размер пакета - 16, начальная скорость обучения - 0,01, оптимизатор - стохастический градиентный спуск с моментом 0,937. Сводные метрики моделей представлены в таблице 7.

Таблица 7 — Результаты экспериментального сравнения моделей YOLO на наборе данных RDD 2022

Модель	mAP50	mAP50-95	Точность	Полнота	F1-мера
YOLOv26n	0,8190	0,5517	0,8707	0,8279	0,8488
YOLOv12n	0,8128	0,5376	0,8564	0,8094	0,8322
YOLOv8n	0,8090	0,5333	0,8392	0,8315	0,8353
YOLOv11n	0,8028	0,5302	0,8432	0,8213	0,8321

Анализ результатов показал, что наилучшую среднюю точность mAP50 демонстрирует YOLOv26n (0,8190), который также лидирует по точности (precision - 0,8707), полноте (recall - 0,8279) и F1-мере (0,8488). Данный результат не является неожиданным, поскольку в архитектуре YOLOv26n были внедрены улучшения, направленные на повышение точности детекции: использование ProgLoss и STAL (Spatial and Temporal Aggregation Loss) позволяет эффективнее распознавать малые объекты, что критически важно для задачи детекции трещин и выбоин на дорожном покрытии. Кроме того, оптимизатор MuSGD обеспечивает более стабильное обучение и быструю сходимость.

YOLOv12n занимает второе место по mAP50 (0,8128), однако уступает YOLOv8n по полноте (recall). YOLOv8n показывает сбалансированные показатели точности (precision) и полноты (recall) при минимальном времени обучения. YOLOv11n показал наименьшие результаты среди всех рассматриваемых моделей.

1.3 Выводы

В результате выполнения аналитического обзора определены основные типы дефектов дорожного покрытия, подлежащие фиксации в системе: трещины (продольные, поперечные, сетка, заделанные), выбоины, колеи, дефекты разметки, люки колодцев и заплатки. Установлено, что дефекты дорожного покрытия оказывают существенное влияние на безопасность движения - по данным статистики, только в 2021 году из-за

неудовлетворительного состояния дорог произошло почти 53 тысячи ДТП, в которых погибло 25 тысяч и ранено 68 тысяч человек [8].

Проведён анализ традиционных подходов к обнаружению дефектов (плановые осмотры, передвижные лаборатории, обращения граждан) и информированию водителей (дорожные знаки, навигационные приложения). Выявлены их основные недостатки: неоперативность, неполнота данных, высокие затраты и отсутствие обратной связи. Выполнен обзор существующих программных решений - Яндекс.Карты, 2ГИС, Waze, «Карта убитых дорог». Показано, что ни одно из них не сочетает детальную классификацию дефектов, оперативное оповещение водителей и модерацию сообщений, что подтверждает актуальность разработки собственной системы.

На основе выявленных недостатков сформулированы функциональные и нефункциональные требования к разрабатываемой системе, включая подсистемы управления доступом, управления дефектами, автоматической детекции, клиентское приложение и подсистему мониторинга.

Проведён анализ технологий и инструментов, применимых для разработки. Рассмотрены архитектурные подходы (монолитная и микросервисная архитектура), веб-фреймворки (FastAPI, Django REST, ASP.NET Core, NestJS, Spring Boot), обратные прокси и API-шлюзы (NGINX, HAProxy, Traefik, Envoy), кроссплатформенные фреймворки для мобильных приложений (React Native, Flutter, Xamarin, Ionic, Kotlin Multiplatform), системы управления базами данных (PostgreSQL/PostGIS, Microsoft SQL Server, Oracle Spatial, MongoDB), объектные хранилища (Amazon S3, MinIO, Ceph), in-memory системы (Redis, Memcached, Tarantool) и инструменты мониторинга (Prometheus, Grafana, Alertmanager). Кроме того, проведено экспериментальное сравнение моделей семейства YOLO (YOLOv8n, YOLOv11n, YOLOv12n, YOLOv26n) на наборе данных RDD 2022. Наилучшие показатели точности и полноты продемонстрировала модель YOLOv26n (mAP50 - 0,8190, precision - 0,8707, recall - 0,8279).

Таким образом, задачи аналитического обзора выполнены в полном объёме. Полученные результаты являются основой для проектирования архитектуры системы и обоснованного выбора технологий в следующей главе.

2 Проектирование системы информирования водителей о дефектах дорожного покрытия

2.1 Архитектура системы информирования водителей о дефектах дорог

2.1.1 Контекст взаимодействия системы с пользователями

На верхнем уровне абстракции система взаимодействует с четырьмя ролями пользователей: водитель, информатор, модератор и администратор.

1. **Водитель** - конечный пользователь системы, который просматривает дефекты на карте, получает уведомления о приближении к дефектам, создаёт новые дефекты (статус pending), просматривает и редактирует свои дефекты.
2. **Информатор** - верифицированный пользователь, который фиксирует новые дефекты, редактирует и отслеживает свои сообщения о состоянии дорожного покрытия.
3. **Модератор** - сотрудник, который проверяет достоверность поступивших дефектов, подтверждает или отклоняет их.
4. **Администратор** - сотрудник, управляющий учетными записями и правами доступа.

Диаграмма контекста, иллюстрирующая взаимодействие системы с внешними сущностями, представлена на рисунке 2.

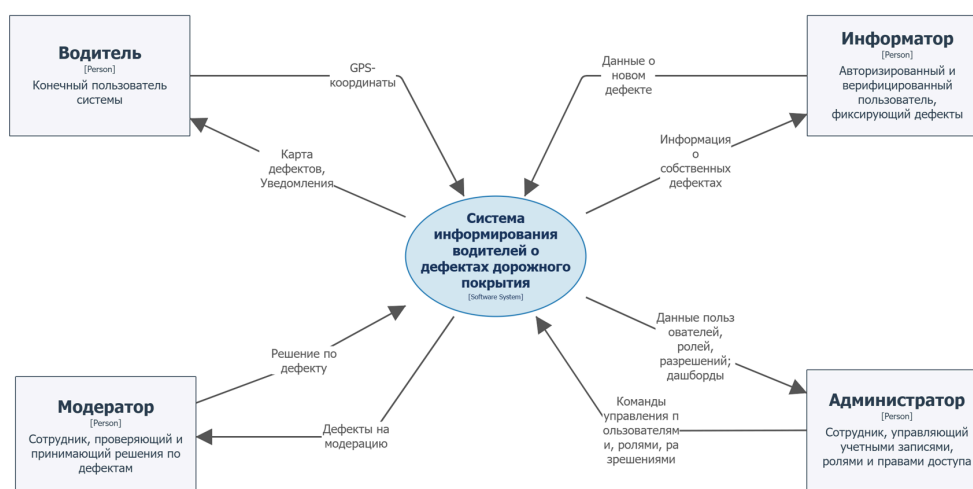


Рисунок 2 — Диаграмма контекста системы

2.1.2 Компоненты системы и выбор технологий для их реализации

Система состоит из следующих основных компонентов. Выбор технологий для каждого компонента обоснован результатами анализа, проведённого в главе 1.

Клиентское приложение реализовано на фреймворке React Native, сравнительный анализ которого выполнен в разделе 1.3.4. Данный выбор обусловлен кроссплатформенностью (единая кодовая база для iOS и Android), богатой экосистемой библиотек для работы с картами и геолокацией, а также большим сообществом.

Обратный прокси (NGINX), анализ которого представлен в разделе 1.3.3, выполняет функции балансировки нагрузки, маршрутизации запросов, ограничения частоты запросов и TLS-терминации. Выбор NGINX обусловлен его высокой производительностью, низким потреблением ресурсов и широкой распространённостью в высоконагруженных системах.

Сервис управления доступом реализован на веб-фреймворке FastAPI. В разделе 1.3.2 приведены результаты нагрузочного тестирования, которые подтверждают, что FastAPI показывает наилучшую производительность среди рассмотренных аналогов. Сервис отвечает за регистрацию, аутентификацию, управление сессиями, ролями и разрешениями.

Сервис автоматической детекции дефектов (FastAPI + YOLO) предназначен для автоматической модерации дефектов. Для детекции выбрана модель YOLOv26n, экспериментальное сравнение которой с другими версиями семейства YOLO представлено в разделе 1.3.9. Сервис взаимодействует с сервисом управления дефектами асинхронно через очередь сообщений Redis Streams.

База данных пользователей (PostgreSQL) хранит данные аккаунтов пользователей, роли и права доступа.

База данных дефектов (PostgreSQL + PostGIS) хранит геоданные о дефектах дорожного покрытия. Преимущества использования PostGIS для хранения геопространственных данных, включая поддержку стандартов

OGC, пространственных индексов GiST и полную поддержку ACID, обоснованы в разделе 1.3.5.

Хранилище сессий и кэша (Redis), анализ которого выполнен в разделе 1.3.7, обеспечивает временное хранение сессий пользователей и часто запрашиваемых данных. Redis выбран благодаря высокой производительности, поддержке TTL для автоматического удаления устаревших данных.

Очередь сообщений (Redis Streams) используется для асинхронного взаимодействия между сервисом управления дефектами и сервисом автоматической детекции. При создании нового дефекта сервис управления дефектами публикует сообщение в поток `defect:detection`. Механизм `consumer groups` обеспечивает гарантию доставки `at least once` и позволяет горизонтально масштабировать сервис детекции - несколько независимых инстансов могут работать одновременно, распределяя сообщения между собой. При выходе из строя одного из инстансов его неподтверждённые сообщения перераспределяются между работающими инстансами.

Хранилище файлов (MinIO) используется для хранения фотографий дефектов и картографических тайлов. Как отмечается в разделе 1.3.6, MinIO представляет собой S3-совместимое объектное хранилище с открытым исходным кодом, предназначенное для развёртывания в собственной инфраструктуре.

Агрегатор метрик (Prometheus), устройство которого подробно описано в разделе 1.3.8, обеспечивает сбор и хранение временных рядов метрик от сервисов, используя `pull`-модель, предпочтительную для микросервисной архитектуры.

Панель мониторинга (Grafana), также рассмотренная в разделе 1.3.8, обеспечивает визуализацию метрик и создание дашбордов для администратора, поддерживая Prometheus в качестве источника данных.

Диаграмма контейнеров в нотации C4 представлена на рисунке 3.

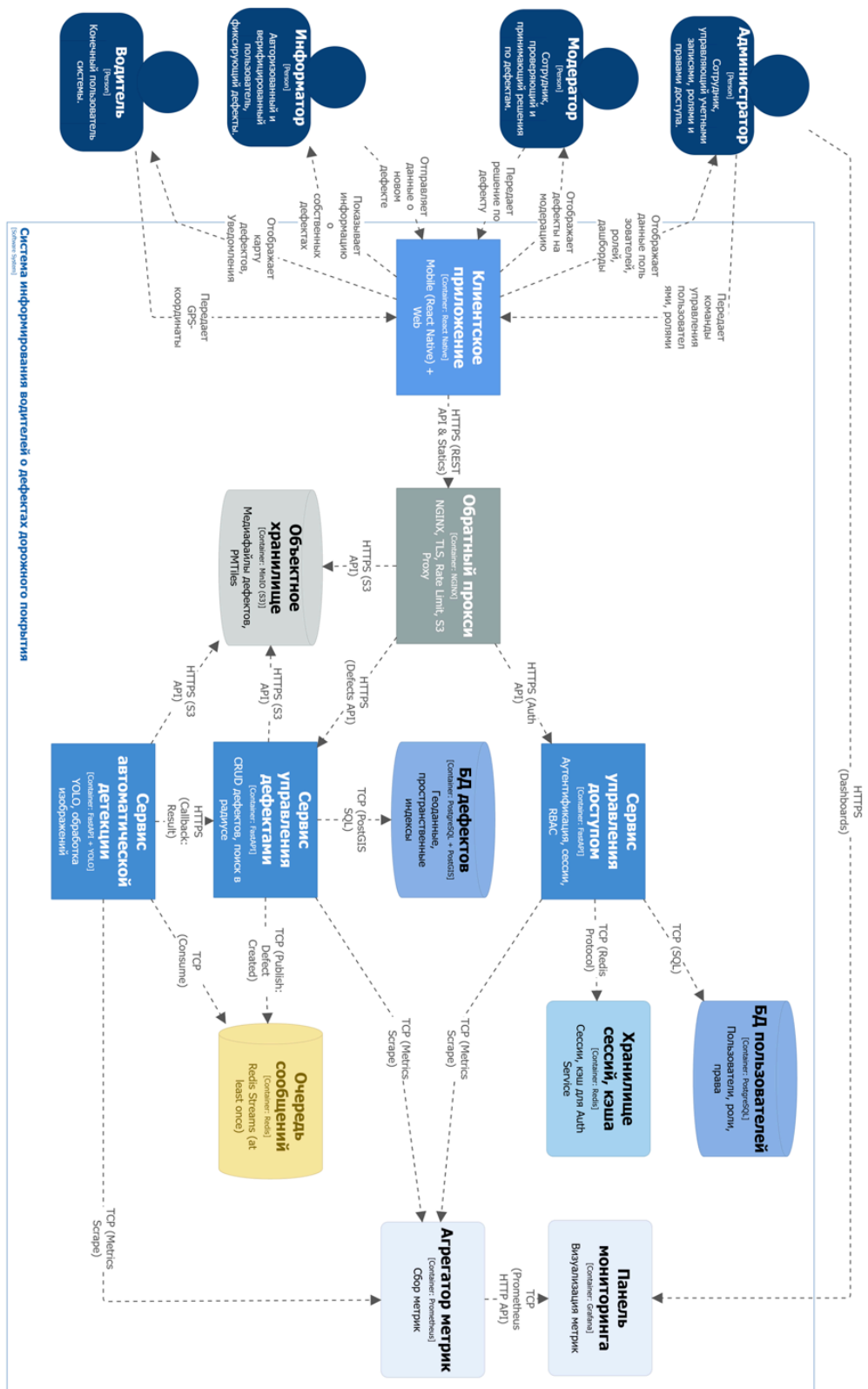


Рисунок 3 — Диаграмма контейнеров в нотации C4

2.1.3 Модель данных

2.1.3.1 Концептуальная модель

На концептуальном уровне система оперирует следующими основными сущностями:

1. **Пользователь** - субъект, взаимодействующий с системой (водитель, модератор, администратор).
2. **Сессия** - фиксирует факт входа пользователя в систему.
3. **Роль** - группа прав доступа (user, moderator, admin).
4. **Разрешение** - конкретное действие, разрешённое пользователю.
5. **Дефект** - основная сущность системы, описывающая повреждение дорожного покрытия.
6. **Дорога** - элемент дорожной сети, импортированный из OSM-данных.
7. **История входов** - аудит действий пользователя по входу в систему.

Связи между сущностями:

1. пользователь имеет несколько сессий и записей в истории входов;
2. пользователю могут быть назначены несколько ролей;
3. роль может содержать несколько разрешений;
4. пользователь создаёт и модерирует дефекты;
5. дефект привязан к дороге.

Концептуальная модель данных представлена на рисунке 4.

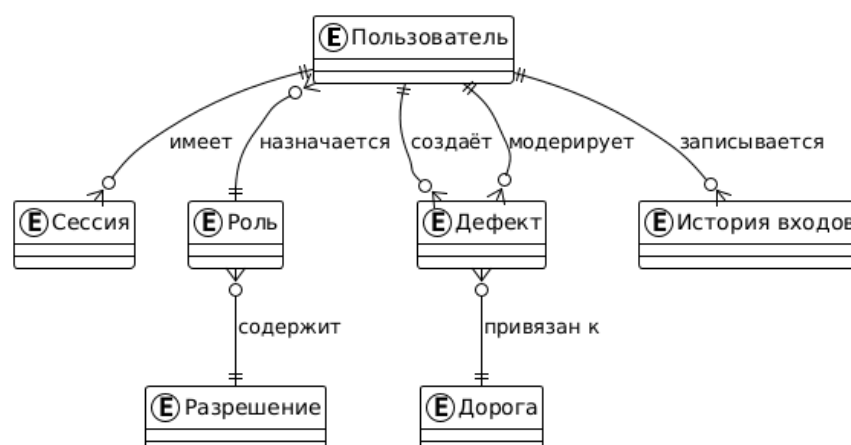


Рисунок 4 — Концептуальная модель данных

2.1.3.2 Логическая модель данных

Сервис управления доступом:

База данных сервиса управления доступом включает таблицы `users`, `roles`, `permissions`, `user_roles`, `role_permissions` и `login_history`. Связи между таблицами:

1. пользователи и роли - связь многие-ко-многим (через `user_roles`);
2. роли и разрешения - связь многие-ко-многим (через `role_permissions`);
3. пользователи и история входов - связь один-ко-многим.

Для хранения сессий используется Redis. Сессии хранятся с TTL, соответствующим сроку действия refresh-токена. ER-диаграмма сервиса управления доступом представлена на рисунке 5.

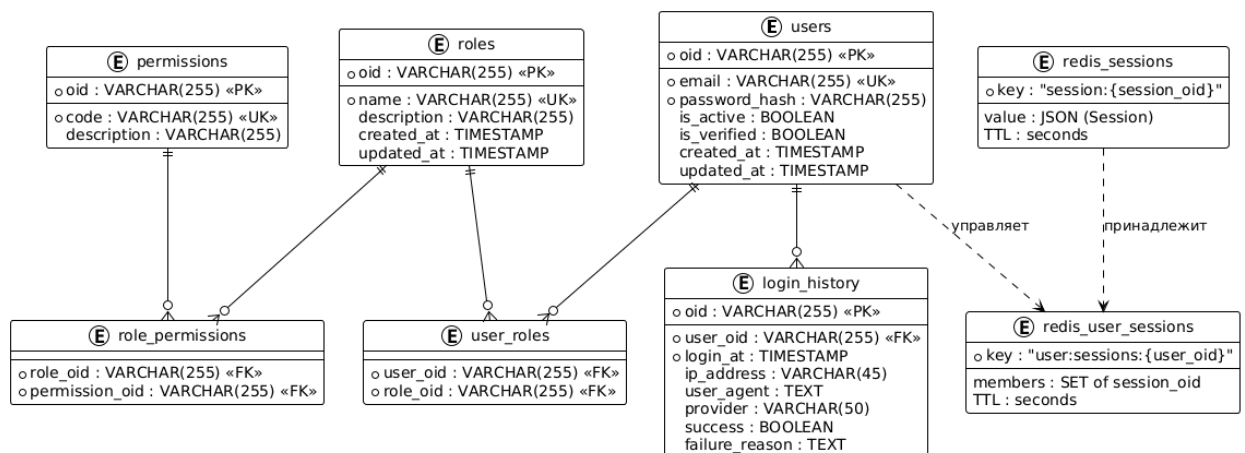


Рисунок 5 — ER-диаграмма сервиса управления доступом

Сервис управления дефектами:

База данных сервиса управления дефектами использует PostgreSQL с расширением PostGIS. Основная таблица **`road_defects`** содержит:

1. геометрические поля: `original_geometry` (исходные координаты), `snapped_geometry` (привязанные к дороге);
2. атрибуты дефекта: `defect_type` (тип), `severity` (уровень опасности), `description` (описание);
3. статусную модель: `status` (`pending`, `approved`, `rejected`, `fixed`), `rejection_reason`;
4. привязку к дороге: `osm_way_id`, `road_name`, `road_class`;

5. фотографии: поле photos (JSON-массив ссылок на файлы в MinIO);
6. аудиторскую информацию: created_by, created_at, moderated_by, moderated_at.

Для повышения производительности пространственных запросов созданы GiST-индексы на полях original_geometry и snapped_geometry, а также B-tree индекс на поле status.

Дорожная сеть импортируется из OSM-файлов (.osm.pbf) и хранится в таблицах, сгенерированных инструментом osm2pgsql:

1. **planet_osm_line** - содержит линейные объекты (дороги, реки, железные дороги);
2. **planet_osm_roads** - содержит только дороги (упрощённый вариант planet_osm_line);
3. **planet_osm_point** - содержит точечные объекты (POI, здания, достопримечательности);
4. **planet_osm_polygon** - содержит полигональные объекты (здания, парки, водоёмы);
5. **planet_osm_nodes**, **planet_osm_ways**, **planet_osm_rels** - служебные таблицы для хранения топологии OSM-данных.

Таблицы дорожной сети были импортированы один раз при инициализации системы и не изменяются в процессе её работы. Для пространственных запросов (привязка дефекта к дороге) используется таблица planet_osm_line с индексом на геометрическом поле.

ER-диаграмма сервиса управления дефектами представлена на рисунке 6.

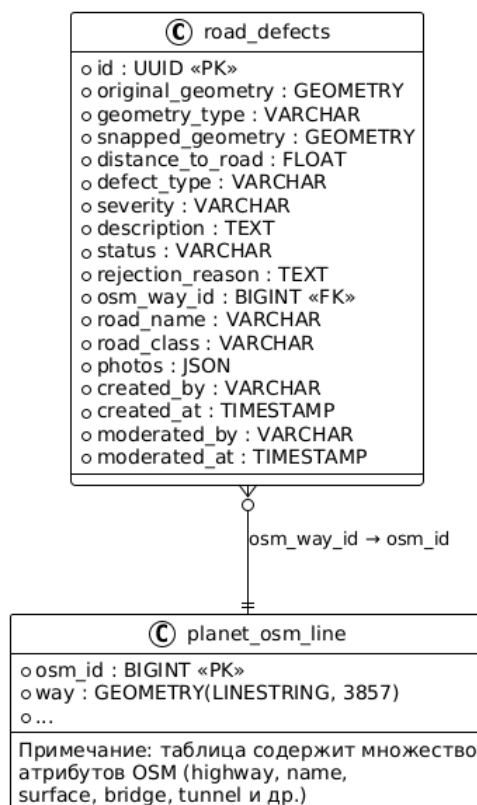


Рисунок 6 — ER-диаграмма сервиса управления дефектами

2.2 Описание ключевых алгоритмов для работы с дефектами дорог

В данном разделе рассматриваются алгоритмы, определяющие специфическую функциональность системы: загрузку и отображение дефектов на карте, динамическое оповещение водителя с учётом скорости и направления движения, а также автоматическую модерацию дефектов с использованием компьютерного зрения. Остальные алгоритмы (аутентификация, обновление токенов, проверка прав доступа, создание дефекта, поиск в радиусе и в видимой области карты) вынесены в приложения А и Б.

2.2.1 Загрузка и отображение дефектов на карте

Отображение дефектов на интерактивной карте является ключевой функцией клиентского приложения. Для обеспечения плавной работы при перемещении и масштабировании карты используется механизм интеллектуальной загрузки дефектов с разбиением на сетку и кэшированием.

Подготовка картографической подложки: картографическая подложка представляет собой векторный тайловый набор, размещённый в объектном хранилище. Такой формат позволяет хранить всю карту в одном файле и получать необходимые тайлы по HTTP-запросам с поддержкой range-запросов, что даёт возможность работать с большими объёмами данных без полной загрузки файла.

Сеточная структура хранения дефектов: дефекты в клиентском приложении индексируются по сетке с размером ячейки 0,01 градуса (приблизительно 1,1 км). Для каждого дефекта вычисляется ключ ячейки по его координатам, что позволяет быстро находить дефекты в заданной области без полного перебора всего массива.

Загрузка дефектов по области экрана: при первом открытии карты или при её перемещении срабатывает механизм отложенной загрузки с задержкой 300 миллисекунд после остановки движения. Тип загрузки зависит от текущего масштаба:

1. При масштабе менее 12 загружаются все дефекты в пределах видимой области экрана одним запросом с адаптивным лимитом (до 200 дефектов при малом масштабе).
2. При масштабе 12 и более загружаются дефекты из расширенной области (видимая область плюс соседние ячейки), лимит увеличивается до 500–1000 дефектов, что обеспечивает плавное перемещение карты без «пустот» на границах.

Кэширование загруженных дефектов: для снижения количества запросов к серверу используется кэширование дефектов по ячейкам с временем жизни 10 минут. При загрузке новой области сначала проверяется наличие данных в кэше. Кэш ограничен по размеру - при превышении 100 ячеек удаляется самая старая.

Форматирование и отображение дефектов: полученные от сервера дефекты проходят этап форматирования: определяется геометрический тип (точка или линия), извлекаются координаты, каждому дефекту присваивается цвет в зависимости от уровня опасности. Отображение выполняется через механизм источников и слоёв картографической библиотеки с поддержкой кластеризации на малом масштабе (менее 14).

```

sequenceDiagram
    participant User
    participant ClientApp as Клиентское приложение
    participant MapMgr as Менеджер сетки и кэша
    participant APIGW as API Gateway (NGINX)
    participant DefectSvc as Сервис управления дефектами
    participant DB as PostgreSQL + PostGIS
    participant CartLib as Картографическая библиотека

    Note over ClientApp: Пользователь
    Note over ClientApp: Открывает карту
    ClientApp->>ClientApp: Запрос геокоординат
    ClientApp->>ClientApp: Определение начальных координат
    ClientApp->>ClientApp: Инициализация карты
    ClientApp->>ClientApp: Карта готова
    ClientApp->>ClientApp: Инициализация менеджера сетки

    Note over ClientApp: Переключение/масштабирование
    ClientApp->>ClientApp: Событие touchend (с задержкой 300 мс)
    ClientApp->>ClientApp: Запрос загрузки для области (bounds, zoom)

    Note over ClientApp: Инициализация карты
    ClientApp->>ClientApp: Запрос геокоординат
    ClientApp->>ClientApp: Определение начальных координат
    ClientApp->>ClientApp: Инициализация карты
    ClientApp->>ClientApp: Карта готова
    ClientApp->>ClientApp: Инициализация менеджера сетки

    Note over ClientApp: Определение области загрузки
    ClientApp->>ClientApp: Событие touchend (с задержкой 300 мс)
    ClientApp->>ClientApp: Запрос загрузки для области (bounds, zoom)

    Note over ClientApp: Проверка кэша
    ClientApp->>ClientApp: [Масштаб < 12]
    ClientApp->>ClientApp: Режим "viewport" (один запрос)
    ClientApp->>ClientApp: Установка лимита 200 дефектов
    ClientApp->>ClientApp: [Масштаб >= 12]
    ClientApp->>ClientApp: Режим "area" (расширенная область)
    ClientApp->>ClientApp: Расчёт соседних ячеек (радиус 1)
    ClientApp->>ClientApp: Установка лимита 500-1000 дефектов

    Note over ClientApp: Форматирование и отображение
    ClientApp->>ClientApp: Сбор уникальных дефектов из всех ячеек
    ClientApp->>ClientApp: Форматирование дефектов
    ClientApp->>ClientApp: [Форматирование:
    - Определение геометрического типа (точка или линия)
    - Извлечение координат
    - Присвоение цвета по уровню опасности]
    ClientApp->>ClientApp: Передача отформатированных дефектов
    ClientApp->>ClientApp: Обновление источников данных
    ClientApp->>ClientApp: Отображение обновлено
    ClientApp->>ClientApp: Дефекты отображены на карте

    Note over ClientApp: Дефекты отображены на карте
    ClientApp->>ClientApp: Дефекты отображены на карте
  
```

2.2.2 Оповещение водителя при приближении к дефекту

Определение параметров движения:

40

$$\theta = \text{atan2}(\sin(\Delta\lambda) \cos(\varphi_2), \cos(\varphi_1) \sin(\varphi_2) - \sin(\varphi_1) \cos(\varphi_2) \cos(\Delta\lambda)) \quad (1)$$

где φ_1, λ_1 - широта и долгота предыдущей точки, φ_2, λ_2 - широта и долгота текущей точки, $\Delta\lambda = \lambda_2 - \lambda_1$ - разность долгот.

Расчёт динамического радиуса оповещения:

Радиус оповещения рассчитывается динамически в зависимости от скорости:

$$R = \min(\max(R_{\text{base}} + vk, R_{\text{min}}), R_{\text{max}}) \quad (2)$$

где $R_{\text{base}} = 80$ м - базовый радиус, v - скорость (км/ч), $k = 1, 5$ - коэффициент, $R_{\text{min}} = 50$ м, $R_{\text{max}} = 300$ м.

Проверка направления движения:

При движении со скоростью более 5 км/ч дополнительно проверяется, находится ли дефект впереди автомобиля. Угол между направлением движения и направлением на дефект вычисляется по формуле:

$$\alpha = |\theta_{\text{defect}} - \theta| \quad (3)$$

где θ_{defect} - азимут от текущего положения пользователя к дефекту, θ - текущее направление движения.

Уведомление отправляется только если угол $\alpha \leq 45^\circ$ (дефект находится в конусе обзора перед автомобилем).

Фильтрация и отправка уведомлений:

Поиск дефектов выполняется с использованием сетчатого индекса - определяются ячейки, попадающие в радиус поиска, и собираются все дефекты из этих ячеек. Отфильтровываются дефекты, не соответствующие настройкам пользователя, а также дефекты, уведомление о которых уже было отправлено в течение последней минуты. Для каждого подходящего дефекта вычисляется примерное время до столкновения:

$$t = \frac{\frac{d}{1000}}{\frac{v}{3600}} \quad (4)$$

(при $v < 1$ км/ч время не вычисляется).

Уведомление формируется с учётом уровня опасности - для критических дефектов используется более интенсивная вибрация.

Диаграмма последовательности оповещения водителя представлена на рисунке 8.

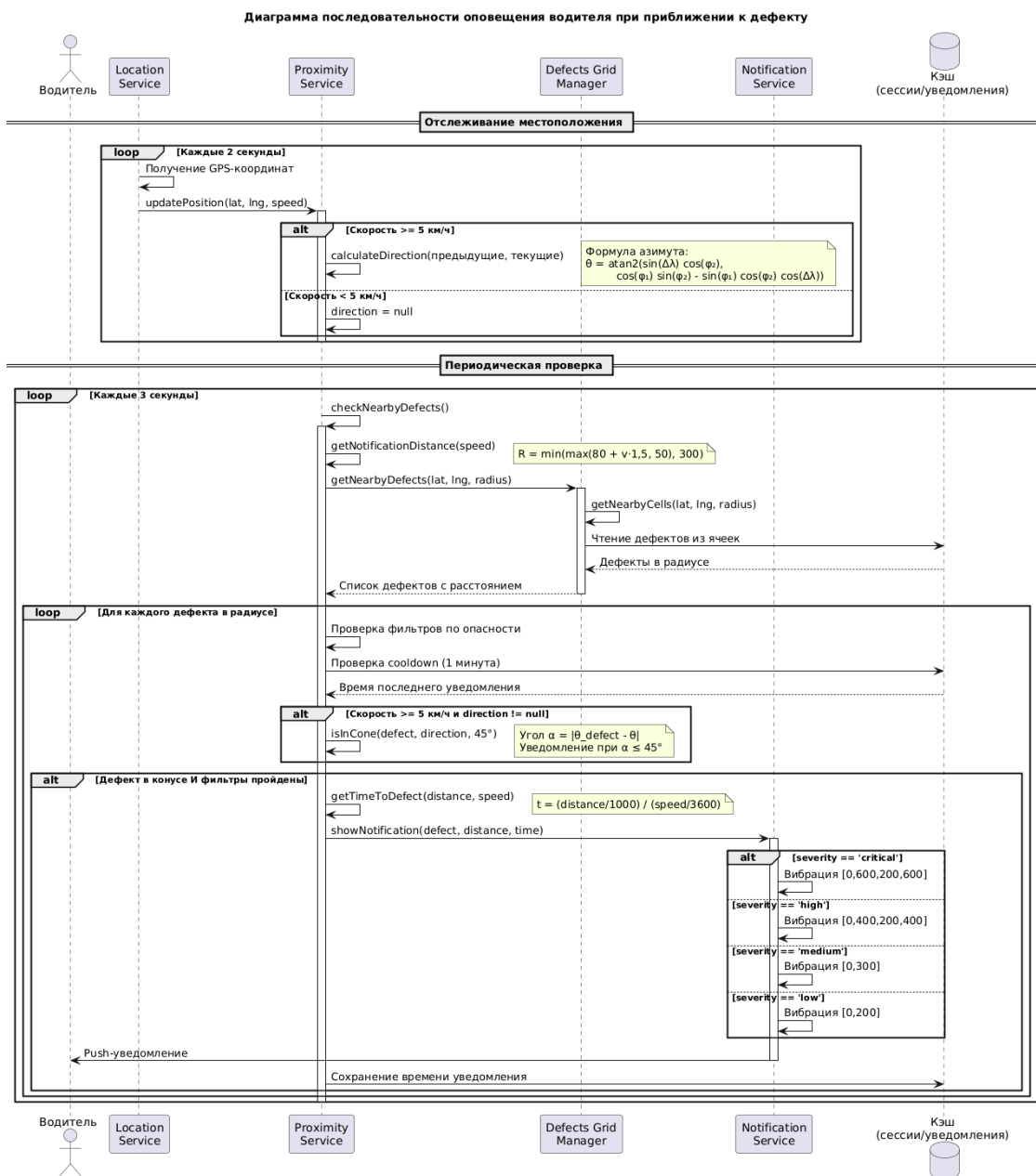


Рисунок 8 — Диаграмма последовательности оповещения водителя

2.2.3 Создание дефекта

Создание нового дефекта является одной из основных функций системы, доступной авторизованным и верифицированным пользователям.

Процесс включает сбор геометрических данных (точек на карте), привязку к дорожной сети, добавление фотографий и отправку на модерацию.

Режим рисования на карте: пользователь переводит карту в режим добавления дефекта. При кликах на карте фиксируются точки. Если добавлена одна точка, дефект считается точечным; если две и более - линейным (протяжённым). Каждая добавленная точка проходит привязку к дорожной сети с использованием пространственного запроса к серверу.

Привязка к дорожной сети: при добавлении каждой точки клиент отправляет запрос к сервису управления дефектами с координатами точки. Сервер выполняет поиск ближайшей дороги в радиусе 15 метров с помощью функций PostGIS (ST_DWithin, ST_ClosestPoint). Если дорога найдена, точка привязывается к ближайшей точке на дороге, и пользователю отображается название дороги. Если дорога не найдена в заданном радиусе, точка не принимается, и пользователю выводится сообщение об ошибке.

Добавление фотографий: пользователь может сделать фотографию дефекта на камеру устройства или выбрать из галереи. Для загрузки фотографий на сервер используется объектное хранилище MinIO через S3-совместимый API. Каждый дефект может сопровождаться несколькими фотографиями.

Заполнение атрибутов дефекта: после завершения рисования пользователь заполняет форму, в которой указывает:

1. тип дефекта (продольная трещина, поперечная трещина, сетка трещин, заделанная трещина, выбоина, смытая разметка, люк колодца, заплатка, колейность и др.);
2. уровень опасности (низкий, средний, высокий, критический);
3. текстовое описание (необязательно).

Отправка на модерацию: после заполнения формы и добавления фотографий пользователь отправляет дефект на сервер. Дефект сохраняется в базе данных со статусом «pending» (на проверке). Сервис управления дефектами публикует сообщение в очередь Redis Streams для автоматической детекции.

Диаграмма последовательности создания дефекта представлена на рисунке 9.

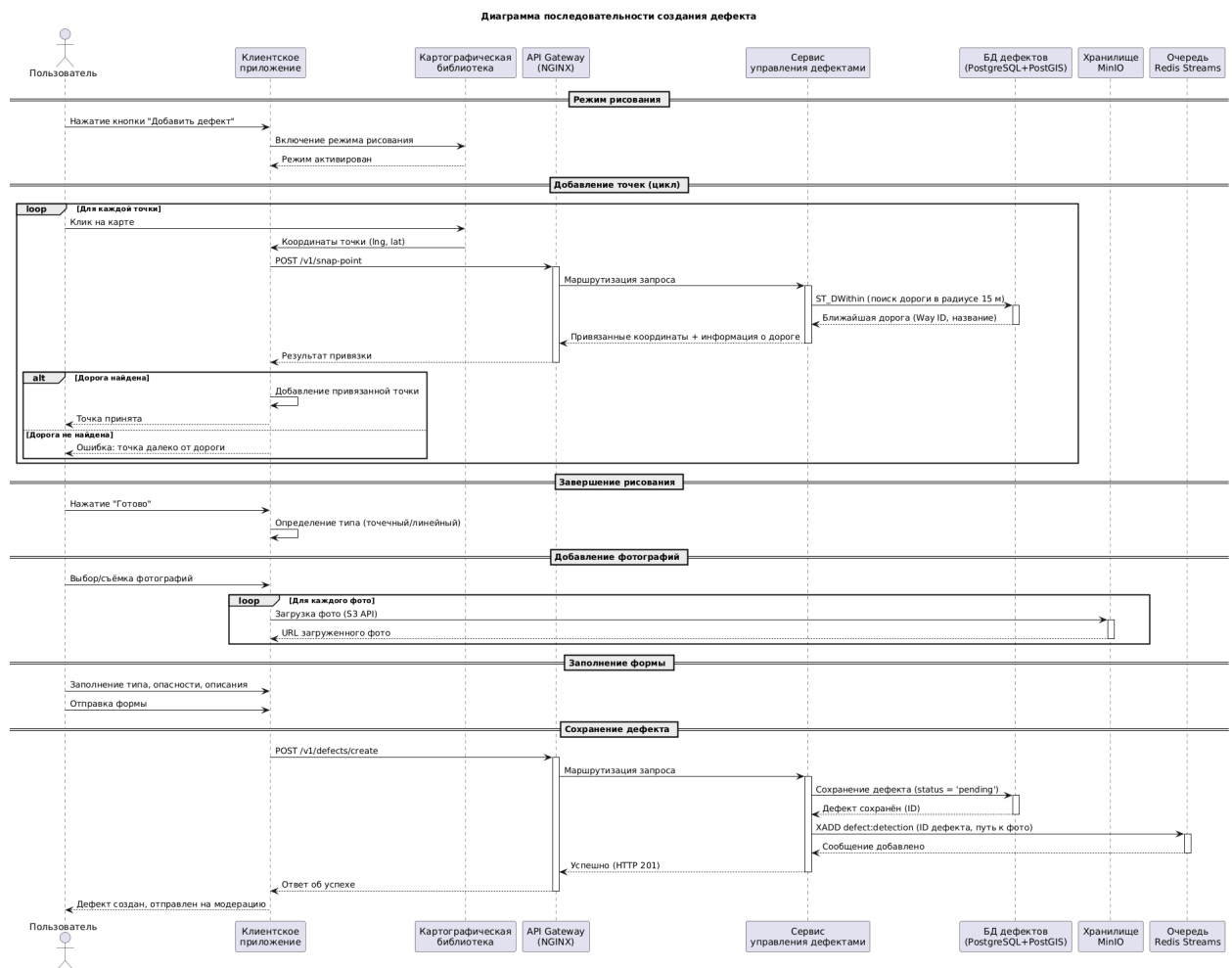


Рисунок 9 — Диаграмма последовательности создания дефекта

2.3 Выводы

В результате проектирования разработана архитектура системы информирования водителей о дефектах дорожного покрытия. Выбрана микросервисная архитектура, включающая сервис управления доступом, сервис управления дефектами, сервис автоматической детекции, клиентское приложение, обратный прокси NGINX и подсистему мониторинга. Данный подход обеспечивает независимое масштабирование компонентов и устойчивость к отказам.

Определён стек технологий для реализации: FastAPI для серверной части, React Native для клиентского приложения, PostgreSQL с расширением PostGIS для хранения геоданных, MinIO для хранения фотографий, Redis для кэширования, сессий и организации очереди сообщений, NGINX для маршрутизации и балансировки нагрузки, Prometheus и Grafana для мониторинга.

Разработана концептуальная и логическая модели данных. Для сервиса управления доступом спроектированы таблицы `users`, `roles`, `permissions`, `user_roles`, `role_permissions` и `login_history`. Для сервиса управления дефектами спроектирована таблица `road_defects` с пространственными полями и GiST-индексами для оптимизации геозапросов.

Разработаны и описаны ключевые алгоритмы. Алгоритм загрузки и отображения дефектов на карте использует сеточный индекс, адаптивное лимитирование и кэширование с временем жизни 10 минут, а также кластеризацию маркеров на малом масштабе. Алгоритм оповещения водителя при приближении к дефекту рассчитывает динамический радиус оповещения от 50 до 300 метров в зависимости от скорости, проверяет направление движения в конусе 45° и фильтрует уведомления по уровню опасности с учётом минутного `cooldown`. Алгоритм создания дефекта включает режим рисования на карте, привязку точек к дорожной сети с помощью функций PostGIS, добавление фотографий в объектное хранилище и отправку дефекта на модерацию через очередь сообщений.

Построены диаграммы контекста, контейнеров в нотации C4 и диаграммы последовательности для всех ключевых сценариев использования: вход, обновление токенов, проверка прав, создание дефекта, поиск дефектов в радиусе и в видимой области карты, загрузка и отображение дефектов на карте, оповещение водителя.

Спроектирована асинхронная обработка дефектов через очередь Redis Streams и возможностью горизонтального масштабирования сервиса детекции дефектов. При выходе из строя одного из инстансов его неподтверждённые сообщения перераспределяются между работающими инстансами с помощью механизма XCLAIM.

Все принятые проектные решения направлены на обеспечение производительности, масштабируемости и надёжности разрабатываемой системы. Спроектированная архитектура полностью соответствует требованиям, сформулированным в главе 1, и является основой для реализации системы в главе 3.

3 Реализация и тестирование системы информирования водителей о дефектах дорожного покрытия

На основе спроектированной архитектуры и принятых проектных решений, описанных в главе 2, была осуществлена практическая реализация всех компонентов системы. В данной главе описываются особенности программирования, конфигурирования и интеграции разработанных модулей, вопросы развёртывания системы, а также приводятся результаты функционального и нагрузочного тестирования, оценка качества модели компьютерного зрения и анализ достижения поставленной цели.

3.1 Реализация сервиса управления доступом

Сервис управления доступом реализован на языке Python с использованием веб-фреймворка FastAPI. Для хранения данных используется PostgreSQL, для управления сессиями - Redis. Аутентификация построена на JWT-токенах с алгоритмом RS256, пароли хэшируются с помощью bcrypt.

Структура проекта: проект организован по принципам чистой архитектуры (Clean Architecture) с разделением на слои: domain (бизнес-сущности и правила), application (use cases), infrastructure (реализация репозиторий и сервисов) и presentation (API handlers). Такой подход обеспечивает независимость бизнес-логики от внешних зависимостей и упрощает тестирование.

Особенности реализации:

1. Для генерации и верификации JWT-токенов использована библиотека authx, настроенная на алгоритм RS256. Приватный и публичный ключи передаются через переменные окружения в base64-кодировке.
2. Сессии пользователей хранятся в Redis с TTL, равным сроку действия refresh-токена (7 дней). Для каждого пользователя поддерживается список активных сессий.
3. Регистрация пользователя автоматически назначает ему роль «user». При успешной регистрации на указанный email

отправляется письмо с ссылкой для верификации, содержащей JWT-токен.

4. Управление ролями и разрешениями реализовано через RBAC (Role-Based Access Control). Администратор может создавать роли, назначать им разрешения и назначать роли пользователям.

Скриншоты документации API сервиса управления доступом вынесены в приложение В.

3.2 Реализация сервиса управления дефектами

Сервис управления дефектами реализован на FastAPI с использованием PostgreSQL и расширения PostGIS для работы с геоданными. Для хранения фотографий используется объектное хранилище MinIO.

Импорт OSM-данных: дорожная сеть импортируется из OSM-файлов (.osm.pbf) с помощью утилиты osm2pgsql. Импорт создаёт таблицы planet_osm_line, planet_osm_point, planet_osm_polygon и другие, содержащие геометрию дорог и объектов инфраструктуры. Процесс импорта автоматизирован через CLI-команды.

Привязка дефектов к дорожной сети: при создании дефекта координаты пользователя привязываются к ближайшей дороге с помощью пространственного запроса к таблице planet_osm_line. Используются функции PostGIS: ST_DWithin для поиска в радиусе и ST_ClosestPoint для вычисления ближайшей точки на дороге.

Хранение и загрузка фотографий: фотографии загружаются в MinIO через S3-совместимый API. Для каждого дефекта создаётся отдельная директория, фотографии получают уникальные имена. URL загруженных фотографий сохраняются в поле photos таблицы road_defects.

Пространственные запросы: для поиска дефектов в радиусе используется функция ST_DWithin, для поиска в видимой области карты - ST_Intersects с прямоугольным полигоном, созданным через ST_MakeEnvelope. GiST-индексы на полях snapped_geometry и original_geometry обеспечивают высокую производительность запросов.

Скриншоты документации API сервиса управления дефектами вынесены в приложение В.

3.3 Реализация сервиса автоматической детекции дефектов

Сервис автоматической детекции дефектов реализован на языке Python с использованием веб-фреймворка FastAPI и библиотеки Ultralytics YOLO для детекции объектов. Сервис предназначен для автоматической модерации дефектов, находящихся в статусе «на проверке», и взаимодействует с сервисом управления дефектами асинхронно через очередь сообщений Redis Streams.

Модель детекции: в сервисе используется предобученная модель YOLOv26n, дообученная на наборе данных RDD 2022 в течение 100 эпох. Основные метрики модели на тестовой выборке: точность (Precision) — 0,8707, полнота (Recall) — 0,8279, средняя точность mAP50 — 0,8190. Модель способна распознавать пять классов дефектов: продольная трещина, поперечная трещина, сетка трещин, иное повреждение (заплатки, колейность) и выбоина.

Пайплайн предобработки изображений: перед подачей в модель каждое изображение проходит последовательность операций, реализованных с использованием библиотеки OpenCV:

1. изменение размера изображения до 640×640 пикселей с билинейной интерполяцией;
2. применение CLAHE (Contrast Limited Adaptive Histogram Equalization) для улучшения локального контраста, что позволяет сделать трещины и другие дефекты более заметными на фоне дорожного покрытия;
3. двусторонняя фильтрация для удаления шума с сохранением границ дефектов.

На рисунке 10 представлен коллаж с примерами результатов детекции дефектов на тестовых изображениях.

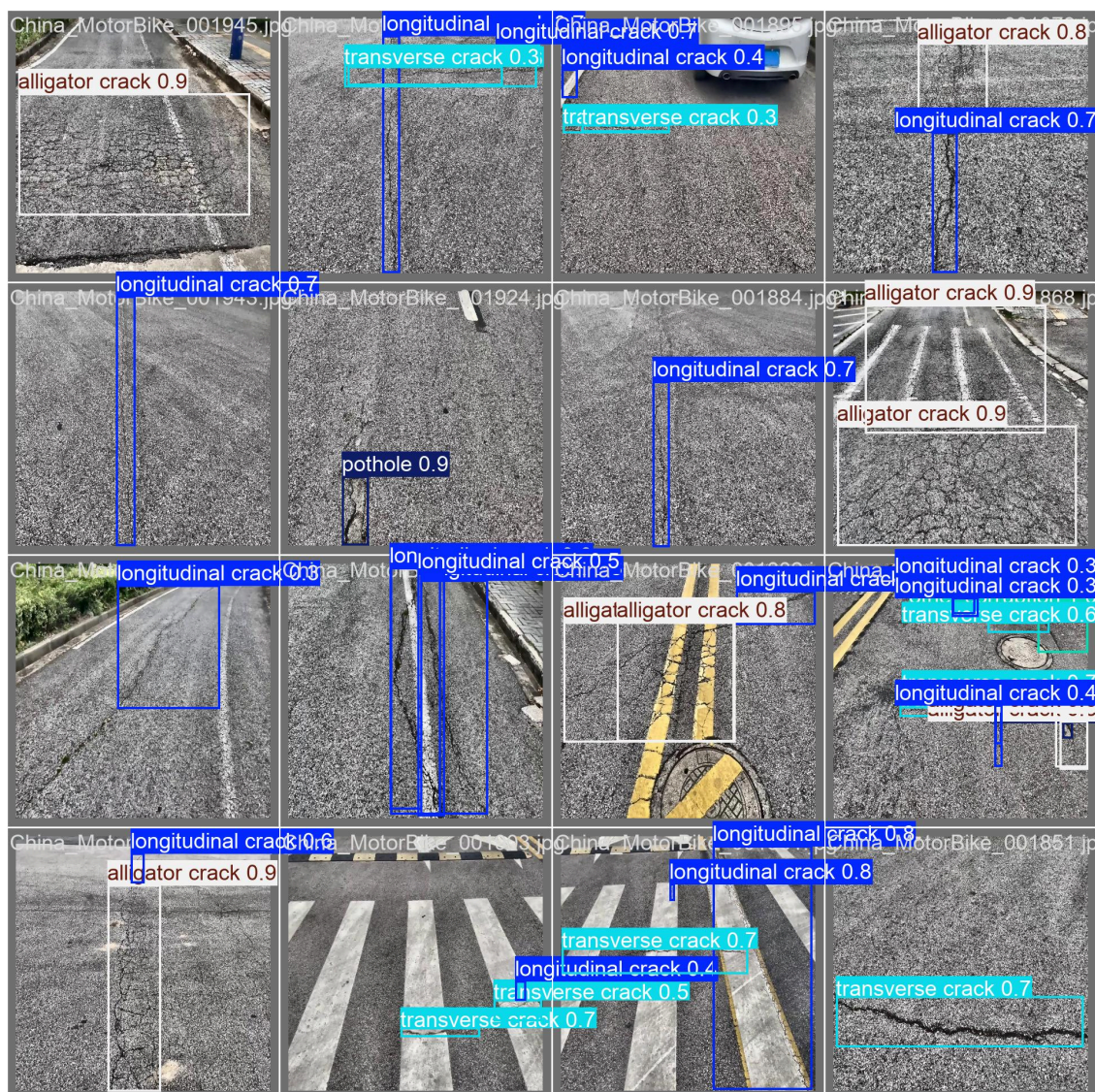


Рисунок 10 — Примеры детекции дефектов дорожного покрытия моделью YOLOv26n

Архитектура и взаимодействие: сервис детекции работает как consumer group в Redis Streams. При создании нового дефекта сервис управления дефектами публикует сообщение в поток defect:detection. Сервис детекции читает сообщения из потока, загружает фотографии из MinIO, выполняет предобработку и детекцию. В случае, если обнаруженный дефект соответствует заявленному типу с уверенностью выше заданного порога, сервис отправляет HTTP-запрос в сервис управления дефектами для автоматического перевода дефекта в статус «подтверждён». В противном случае дефект остаётся в статусе «на проверке» для ручной модерации.

Масштабирование и отказоустойчивость: сервис детекции реализован как несколько независимых инстансов, объединённых в одну consumer group. Redis автоматически распределяет сообщения между

активными инстансами. При выходе из строя одного из инстансов его неподтверждённые сообщения перераспределяются между работающими инстансами с помощью механизма XCLAIM.

3.4 Реализация клиентского приложения

Клиентское приложение разработано на React Native, поддерживает платформы iOS и Android (а также Web через React Native Web). Для отображения карты использована библиотека MapLibre GL, работающая через WebView. Картографическая подложка представлена в формате PMTiles и загружается из MinIO.

Аутентификация и управление сессиями: токены доступа и обновления хранятся в защищённом хранилище (SecureStore на мобильных платформах, localStorage на Web). Реализован механизм автоматического обновления access-токена через интерцептор Axios при получении ошибки 401.

Отображение карты и дефектов: карта инициализируется с начальными координатами, определёнными по геолокации пользователя. Дефекты загружаются с сервера при перемещении карты с использованием сеточного индекса и кэширования. Для отображения дефектов используются два слоя: точечные маркеры (с цветом в зависимости от уровня опасности) и линии для протяжённых дефектов. Реализована кластеризация маркеров при малом масштабе.

Добавление дефекта: пользователь переводит карту в режим рисования, кликами отмечает точки дефекта. Для точечного дефекта достаточно одной точки, для линейного — двух и более. При добавлении каждой точки выполняется привязка к дорожной сети через эндпоинт «/v1/snap-point». После завершения рисования открывается форма с выбором типа дефекта, уровня опасности, описанием и добавлением фотографий.

Оповещения о приближении: сервис proximityService периодически (каждые 3 секунды) проверяет дефекты в динамическом радиусе с учётом скорости и направления движения. При обнаружении дефекта в конусе 45° отправляется push-уведомление.

Скриншоты основных экранов мобильного приложения представлены на рисунках 11–13.

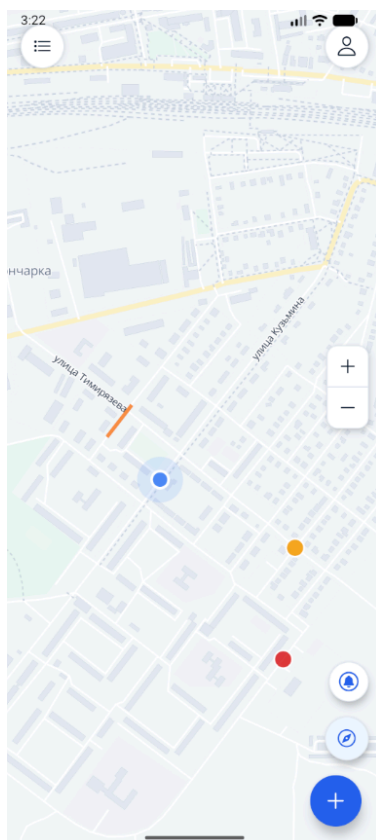


Рисунок 11 — Экран карты с дефектами

Рисунок 12 — Форма добавления дефекта

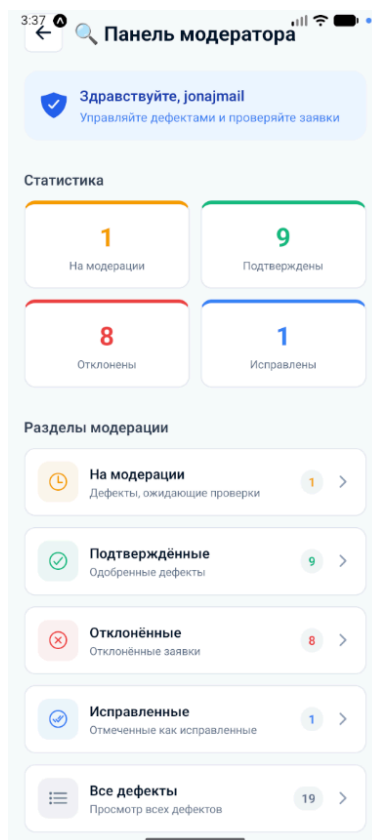


Рисунок 13 — Панель модератора

Остальные скриншоты интерфейса мобильного приложения (окно входа, регистрации, профиль, админ-панель, управление пользователями, ролями, разрешениями, списки дефектов, процесс добавления дефекта, оповещение) вынесены в приложение Г.

3.5 Развёртывание системы информирования водителей о дефектах дорожного покрытия

Развёртывание системы осуществляется с использованием контейнеров Docker, что обеспечивает изоляцию сервисов, воспроизводимость окружения и упрощает установку на различных платформах. Для автоматизации основных операций разработан Makefile, который объединяет часто используемые команды.

Требования к окружению: для работы системы необходим Docker и Docker Compose. Рекомендуемый объём оперативной памяти — не менее 8 ГБ, количество ядер CPU — не менее 4, свободное дисковое пространство — не менее 20 ГБ (для хранения базы данных, фотографий и OSM-данных).

Конфигурация: все настройки сервисов задаются через файл переменных окружения `.env`, который должен быть создан перед запуском. В нём указываются параметры подключения к базам данных, порты, пароли и другие конфигурационные параметры.

Структура сервисов: система включает следующие контейнеры, описанные в файле `docker-compose.yml`:

- `auth-postgres` — база данных для хранения пользователей, ролей и разрешений (PostgreSQL 17.2);
- `auth-redis` — хранилище сессий и кэш для сервиса управления доступом (Redis);
- `auth-service` — сервис управления доступом (FastAPI);
- `defects-postgres` — база данных для хранения геоданных о дефектах с расширением PostGIS (PostGIS 15–3.4);
- `defects-redis` — очередь сообщений и кэш для сервиса дефектов (Redis);
- `defects-minio` — объектное хранилище для фотографий дефектов и картографических тайлов (MinIO);
- `defects-service` — сервис управления дефектами (FastAPI);
- `defects-detection-service` — сервис автоматической детекции дефектов (YOLOv26n);
- `nginx` — обратный прокси для маршрутизации и балансировки нагрузки;
- `prometheus` — агрегатор метрик;
- `grafana` — панель визуализации метрик;
- `tilemaker` (опционально, запускается отдельным профилем) — генератор векторных тайлов из OSM-данных.

Порядок развёртывания: процесс развёртывания включает несколько последовательных этапов. На первом этапе необходимо клонировать репозиторий с исходным кодом системы и создать файл конфигурации `.env` на основе шаблона, заполнив его параметрами подключения к базам данных, секретными ключами и сетевыми портами. Затем следует загрузить OSM-данные для выбранного региона (например, Московской области) и импортировать их в PostGIS. После этого

выполняется сборка и запуск всех контейнеров Docker, применение миграций базы данных с использованием Alembic и инициализация сервиса аутентификации (создание начальных ролей и учётной записи администратора). После выполнения всех шагов система становится доступной для использования. Для проверки работоспособности достаточно открыть в браузере документацию API сервиса управления доступом и сервиса управления дефектами.

Клиентское приложение: мобильное приложение на React Native требует отдельной сборки под целевые платформы (Android и iOS). Картографическая подложка в формате PMTiles предварительно загружается в MinIO.

3.6 Тестирование системы информирования водителей о дефектах дорожного покрытия

Тестирование разработанной системы включало оценку качества сервиса автоматической детекции дефектов, функциональное тестирование клиентского приложения и нагрузочное тестирование серверной части.

3.6.1 Оценка качества сервиса автоматической детекции дефектов

Для оценки качества сервиса автоматической детекции дефектов использовалась обученная модель YOLOv26n. Тестирование проводилось на отдельной тестовой выборке, которая не участвовала в процессе обучения модели. Тестовая выборка сформирована из набора данных RDD 2022 и включает размеченные изображения дорожных дефектов с указанием классов объектов и их координат в виде ограничивающих рамок (bounding boxes).

Для оценки качества использовались стандартные метрики: точность (precision) и полнота (recall).

Точность (Precision) характеризует долю правильно обнаруженных объектов среди всех объектов, которые модель идентифицировала как дефекты:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (5)$$

где «TP» (True Positives) — количество правильно обнаруженных дефектов, «FP» (False Positives) — количество ложных срабатываний.

Полнота (Recall) характеризует долю правильно обнаруженных объектов среди всех реально существующих дефектов на изображении:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (6)$$

где «FN» (False Negatives) — количество пропущенных дефектов.

По результатам оценки на тестовой выборке модель YOLOv26n достигла следующих показателей:

- mAP50 — 0,8190;
- mAP50-95 — 0,5517;
- Precision — 0,8707;
- Recall — 0,8279;
- F1-мера — 0,8488.

Значение точности (Precision = 0,8707) превышает целевой показатель, установленный в цели работы (не ниже 80%). Один инстанс сервиса детекции способен обрабатывать до 30 кадров в секунду (30 FPS), что гарантирует своевременную обработку фотографий даже при пиковой нагрузке.

3.6.2 Функциональное тестирование

Функциональное тестирование проводилось вручную на эмуляторах Android и физических устройствах. Проверялись как успешные сценарии использования (happy path), так и сценарии с ошибками, граничные случаи, а также обработка некорректных входных данных.

Проверка автоматической детекции и размещения дефектов на карте (успешный сценарий):

- авторизованный пользователь переводит карту в режим рисования;

- отмечает на карте точку (для точечного дефекта) или несколько точек (для линейного дефекта);
- система выполняет привязку координат к дорожной сети с использованием функций PostGIS (ST_DWithin, ST_ClosestPoint);
- пользователь заполняет форму (тип дефекта, уровень опасности, описание) и добавляет фотографии;
- дефект отправляется на сервер и получает статус «на проверке» (pending);
- сервис автоматической детекции обрабатывает фотографии с использованием модели YOLOv26n;
- при обнаружении дефекта с уверенностью выше порога дефект переводится в статус «подтверждён» (approved) и отображается на карте.

Проверка автоматической детекции и размещения дефектов на карте (неуспешные сценарии):

- добавление дефекта без фотографий — форма не отправляется, пользователю выводится сообщение об ошибке;
- добавление дефекта с координатами вне дорожной сети (далее 15 метров от ближайшей дороги) — точка не принимается, выводится сообщение «Дорога не найдена»;
- добавление дефекта с типом, не соответствующим обнаруженному моделью — дефект остаётся в статусе «на проверке» для ручной модерации;
- добавление дефекта неавторизованным пользователем — форма недоступна, требуется вход в систему.

Проверка адресного оповещения водителей:

- при движении пользователя с определённой скоростью система рассчитывает динамический радиус оповещения;
- при попадании подтверждённого дефекта в радиус оповещения система проверяет направление движения (угол между направлением движения и направлением на дефект не более 45°);
- при выполнении обоих условий пользователю отправляется push-уведомление.

Проверка адресного оповещения (граничные случаи):

- при скорости менее 5 км/ч проверка направления движения не выполняется, уведомление не отправляется;
- при нахождении дефекта вне конуса 45° уведомление не отправляется;
- повторное уведомление о том же дефекте не отправляется в течение 1 минуты (cooldown);
- при отсутствии сетевого соединения уведомление не отправляется, данные сохраняются локально для повторной попытки.

Проверка ручной модерации (успешный сценарий):

- пользователь с ролью модератора открывает панель модерации;
- просматривает список дефектов со статусом «на проверке»;
- открывает детальную информацию о дефекте (фотографии, координаты, описание);
- подтверждает дефект (статус меняется на «подтверждён») или отклоняет с указанием причины (статус меняется на «отклонён»).

Проверка ручной модерации (неуспешные сценарии):

- модерация дефекта пользователем без роли модератора — недоступно, возвращается ошибка доступа;
- отклонение дефекта без указания причины — форма не отправляется, требуется указать причину;
- модерация уже обработанного дефекта (повторная попытка) — возвращается ошибка, дефект не изменяется.

Дополнительные проверенные сценарии:

- регистрация с некорректным email (без «@») — форма не отправляется, сообщение об ошибке;
- вход с неверным паролем — возвращается ошибка аутентификации, доступ не предоставлен;
- просмотр и редактирование чужих дефектов — недоступно, возвращается ошибка доступа.

Выявленные в процессе тестирования ошибки (некорректная привязка к дорожной сети при определённых углах, отсутствие обработки ошибок

при загрузке фотографий, отсутствие валидации формы регистрации) были исправлены. Все проверенные сценарии работают корректно.

3.6.3 Нагрузочное тестирование

Нагрузочное тестирование проводилось с целью оценки производительности серверной части системы под высокой нагрузкой. Для этого использовался инструмент Locust, который эмулировал поведение трёх типов пользователей: анонимный пользователь, водитель и модератор.

Тестирование выполнялось на следующей конфигурации оборудования: процессор AMD Ryzen 5 5600, оперативная память 16 ГБ. Все сервисы системы (сервис управления доступом, сервис управления дефектами, сервис детекции, базы данных, Redis, MinIO, NGINX, Prometheus, Grafana) были развёрнуты на одной машине с использованием Docker.

В тестировании участвовало до 300 одновременно работающих пользователей, которые выполняли следующие операции: загрузка картографических тайлов, получение списка дефектов и дефектов в области видимости, создание дефектов с привязкой к дорожной сети, модерация дефектов. Сервис автоматической детекции в процессе тестирования обрабатывал изображения со скоростью 30 кадров в секунду (30 FPS).

Агрегированные результаты нагрузочного тестирования представлены в таблице 8.

Таблица 8 — Агрегированные результаты нагрузочного тестирования

Показатель	Значение
Общее количество запросов	208 367
Количество ошибок	3 648 (1,75%)
Среднее время ответа	25,64 мс
95-й перцентиль	48 мс
99-й перцентиль	320 мс
Максимальное время ответа	870 мс
RPS (запросов в секунду)	236,6

Среднее время ответа API (25,64 мс) и 95-й перцентиль (48 мс) удовлетворяют нефункциональному требованию о времени ответа не более 1–2 секунд. Система выдержала нагрузку до 300 пользователей,

сохраняя стабильную работу, что подтверждает достаточность выбранной конфигурации оборудования для планируемой нагрузки.

3.7 Выводы

В результате практической реализации были созданы и протестированы все компоненты системы информирования водителей о дефектах дорожного покрытия.

Разработан сервис управления доступом на FastAPI с JWT-аутентификацией (алгоритм RS256), управлением сессиями через Redis и разграничением прав доступа на основе ролей (RBAC). Реализован сервис управления дефектами на FastAPI с интеграцией PostGIS для пространственных запросов, привязкой дефектов к дорожной сети с использованием функций ST_DWithin и ST_ClosestPoint, а также хранением фотографий в объектном хранилище MinIO.

Сервис автоматической детекции на базе модели YOLOv26n, дообученной на наборе данных RDD 2022, обеспечивает автоматическую модерацию дефектов. Модель достигла следующих метрик: точность (Precision) — 0,8707, полнота (Recall) — 0,8279, средняя точность mAP50 — 0,8190. Производительность сервиса составляет 30 кадров в секунду (30 FPS). Взаимодействие с сервисом управления дефектами осуществляется асинхронно через очередь Redis Streams с поддержкой горизонтального масштабирования и отказоустойчивости (consumer groups, XCLAIM).

Разработано клиентское приложение на React Native с отображением карты на базе MapLibre GL и векторных тайлов PMTiles. Реализованы интеллектуальная загрузка дефектов с сеточным индексированием и кэшированием, режим рисования дефектов на карте с привязкой к дорожной сети, а также сервис оповещения водителя с динамическим расчётом радиуса оповещения (50–300 м) и проверкой направления движения (конус 45°). Развёртывание системы осуществляется с использованием Docker и Docker Compose, автоматизация операций обеспечивается Makefile.

Проведено функциональное тестирование, подтвердившее корректную работу успешных сценариев, обработку ошибок и граничных случаев. Нагрузочное тестирование с помощью Locust и мониторинг в

Grafana проводились на конфигурации оборудования: процессор AMD Ryzen 5 5600, оперативная память 16 ГБ. Все сервисы были развёрнуты на одной машине с использованием Docker. Результаты показали, что система выдерживает нагрузку до 300 одновременно работающих пользователей, среднее время ответа API составляет 25,64 мс, 95-й перцентиль — 48 мс, что соответствует нефункциональным требованиям.

ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы была разработана информационная система информирования водителей о дефектах дорожного покрытия, основанная на краудсорсинговом принципе сбора данных от пользователей и автоматической детекции дефектов с использованием компьютерного зрения. Система обеспечивает оперативное выявление, модерацию, визуализацию дефектов и своевременное оповещение участников движения.

Достижение поставленной цели подтверждается решением всех сформулированных задач.

По результатам аналитического обзора: определены основные типы дефектов дорожного покрытия, подлежащие фиксации в системе (трещины, выбоины, колеи, дефекты разметки, люки колодцев, заплатки). Установлено существенное влияние дефектов на безопасность движения. Выявлены недостатки традиционных подходов к обнаружению дефектов и информированию водителей, а также проведён обзор существующих программных решений (Яндекс.Карты, 2ГИС, Waze, «Карта убитых дорог»), показавший, что ни одно из них не сочетает детальную классификацию дефектов, оперативное оповещение водителей и модерацию сообщений. Проведён анализ технологий и инструментов, применимых для разработки, а также экспериментальное сравнение моделей YOLO на наборе данных RDD 2022, по результатам которого для реализации сервиса детекции выбрана модель YOLOv26n. Сформулированы функциональные и нефункциональные требования к разрабатываемой системе.

По результатам проектирования: обоснован выбор микросервисной архитектуры, обеспечивающей независимое масштабирование компонентов и устойчивость к отказам. Определён стек технологий: FastAPI для серверной части, React Native для клиентского приложения, PostgreSQL с расширением PostGIS для хранения геоданных, MinIO для хранения фотографий, Redis для кэширования, сессий и организации очереди сообщений, NGINX для маршрутизации и балансировки нагрузки, Prometheus и Grafana для мониторинга. Разработаны концептуальная и логическая модели данных, спроектированы ER-диаграммы для сервиса управления доступом и сервиса управления

дефектами. Разработаны и описаны ключевые алгоритмы: загрузка и отображение дефектов на карте с использованием сеточного индекса и кэширования, оповещение водителя с динамическим радиусом (50–300 м) и проверкой направления движения (конус 45°), а также создание дефекта с привязкой к дорожной сети.

По результатам практической части: реализован сервис управления доступом на FastAPI с JWT-аутентификацией (RS256), управлением сессиями через Redis и разграничением прав доступа на основе ролей (RBAC). Реализован сервис управления дефектами на FastAPI с интеграцией PostGIS для пространственных запросов, привязкой дефектов к дорожной сети и хранением фотографий в MinIO. Реализован сервис автоматической детекции дефектов на базе YOLOv26n, дообученной на наборе данных RDD 2022, с асинхронным взаимодействием через Redis Streams и возможностью горизонтального масштабирования. Модель достигла точности (Precision) 0,8707, что превышает целевой показатель 80%, и производительности 30 FPS. Разработано клиентское приложение на React Native с отображением карты (MapLibre GL + PMTiles), добавлением дефектов, кэшированием по сетке и оповещениями о приближении. Организовано развёртывание системы с использованием Docker и Docker Compose, автоматизация операций обеспечена Makefile. Проведено функциональное тестирование, подтвердившее корректную работу успешных сценариев, обработку ошибок и граничных случаев. Нагрузочное тестирование с помощью Locust на конфигурации AMD Ryzen 5 5600, 16 ГБ ОЗУ показало, что система выдерживает нагрузку до 300 одновременно работающих пользователей, среднее время ответа API составляет 25,64 мс, 95-й перцентиль — 48 мс, что соответствует нефункциональным требованиям.

Практическая значимость работы состоит в разработке и реализации серверной части (сервисы управления доступом, управления дефектами и автоматической детекции) и клиентского мобильного приложения. Разработанная система может быть использована для оперативного информирования водителей о дефектах дорог, а также в качестве платформы для взаимодействия граждан с дорожными службами и органами контроля.

Направления дальнейшего развития системы:

- расширение набора распознаваемых типов дефектов и улучшение точности модели за счёт дообучения на новых данных;

- организация двустороннего обмена данными с дорожными службами: передача подтверждённых дефектов для ремонта и получение информации об устранении дефектов;

- объединение системы информирования водителей о дефектах дорог, в частности сервиса управления дефектами и сервиса автоматической детекции, с крупными навигационными сервисами с целью расширения пользовательской базы, увеличения объёма собираемых данных и, как следствие, ещё большего повышения уровня информированности участников дорожного движения о дефектах дорожного покрытия;

Таким образом, цель работы достигнута в полном объёме: разработанная система обеспечивает размещение подтверждённых дефектов на интерактивной карте (точность автоматической детекции — 87% при целевом показателе 80%), адресное оповещение водителей с учётом скорости и направления движения, а также совмещение ручной и автоматической модерации. Система соответствует поставленным требованиям и готова к практическому использованию.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Адещенко К. Р. Инструменты кроссплатформенной разработки: сравнительный анализ // Актуальные исследования. 2024. № 12(194). С. 41–49.
2. ГОСТ Р 50597-2017. Дороги автомобильные и улицы. Требования к эксплуатационному состоянию, допустимому по условиям обеспечения безопасности дорожного движения. Методы контроля. — М.: Стандартинформ, 2017. — 28 с.
3. Левин А. А., Семичев С. А., Казакова И. А. Обзор рынка in-memory систем управления базами данных // Вестник Пензенского государственного университета. 2024. № 4. С. 87–89.
4. Ньюмен С. От монолита к микросервисам. — СПб.: БХВ-Петербург, 2021. — 272 с.
5. Ричардсон К. Микросервисы. Паттерны разработки и рефакторинга. — СПб.: Питер, 2019. — 544 с.
6. Саенко А. А., Габов В. С., Журавлёв А. А. Сравнительный анализ систем управления базами данных при реализации распределенного хранилища геоданных // Инженерный вестник Дона. 2025. № 11(131). С. 21–30.
7. Салтанов Д. С., Арсентьева Н. В. Сравнительный анализ архитектурных подходов к разработке REST API для высоконагруженных систем // Форум молодых учёных. 2025. № 6(106). С. 329–338.
8. Сапрыкин С. С., Пак В. В. Влияние состояния дорожного полотна и дорожных условий на безопасность дорожного движения // Вестник магистратуры. 2022. № 3–1(126). С. 6–8.
9. Amazon Web Services. Amazon S3 Documentation [Электронный ресурс]. URL: <https://docs.aws.amazon.com/s3> (дата обращения: 30.04.2025).
10. Ceph Foundation. Ceph Documentation [Электронный ресурс]. URL: <https://docs.ceph.com/> (дата обращения: 30.04.2025).
11. Grafana Labs. Grafana Documentation [Электронный ресурс]. URL: <https://grafana.com/docs> (дата обращения: 30.04.2025).

12. MinIO. MinIO Documentation [Электронный ресурс]. URL: <https://min.io/docs> (дата обращения: 30.04.2025).

13. Prometheus. Prometheus Documentation [Электронный ресурс]. URL: <https://prometheus.io/docs> (дата обращения: 30.04.2025).

ПРИЛОЖЕНИЕ А

Алгоритмы сервиса управления доступом

Диаграмма последовательности входа пользователя представлена на рисунке А.1.

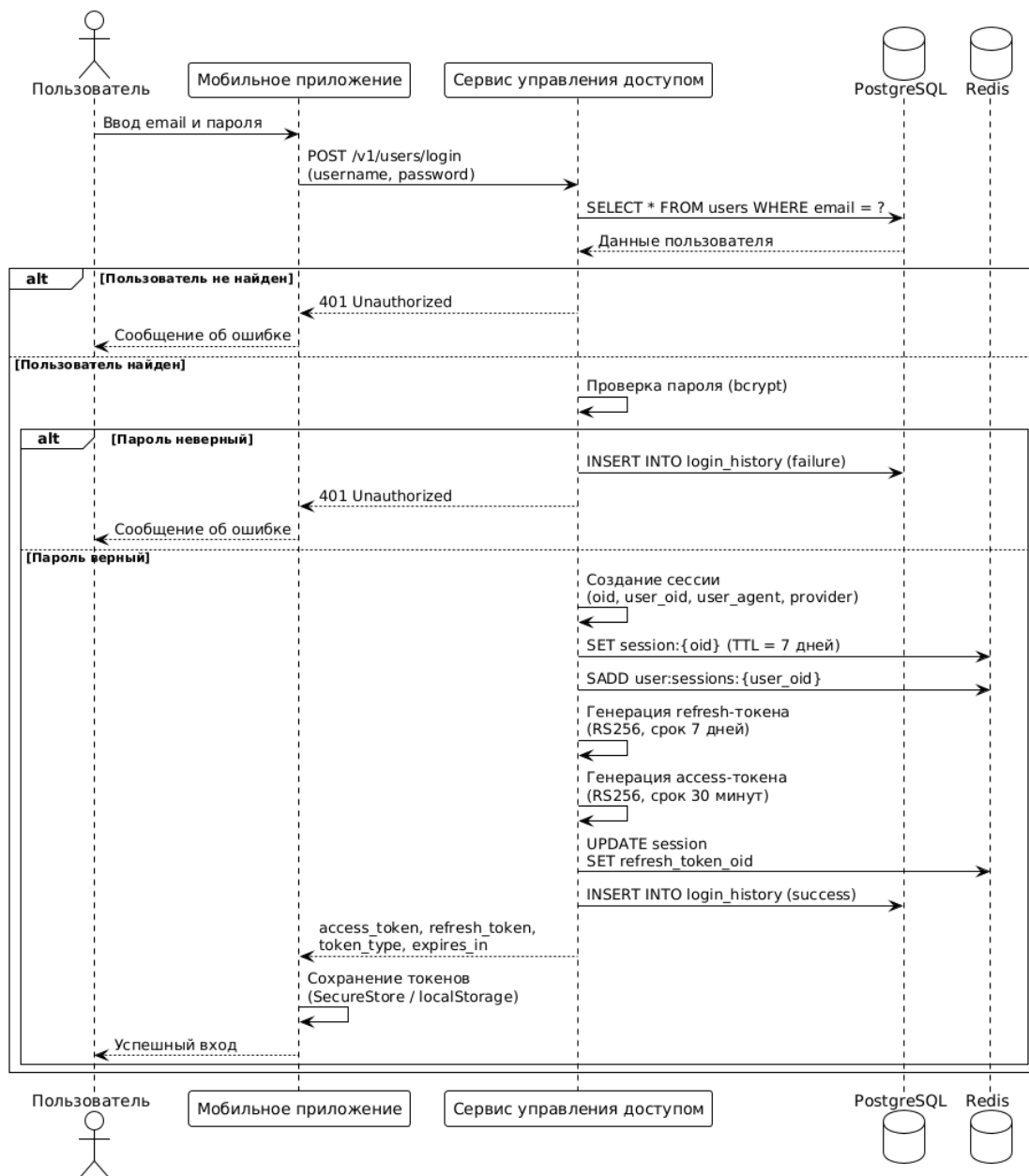


Рисунок А.1 — Диаграмма последовательности входа пользователя

Диаграмма последовательности обновления токенов представлена на рисунке А.2.

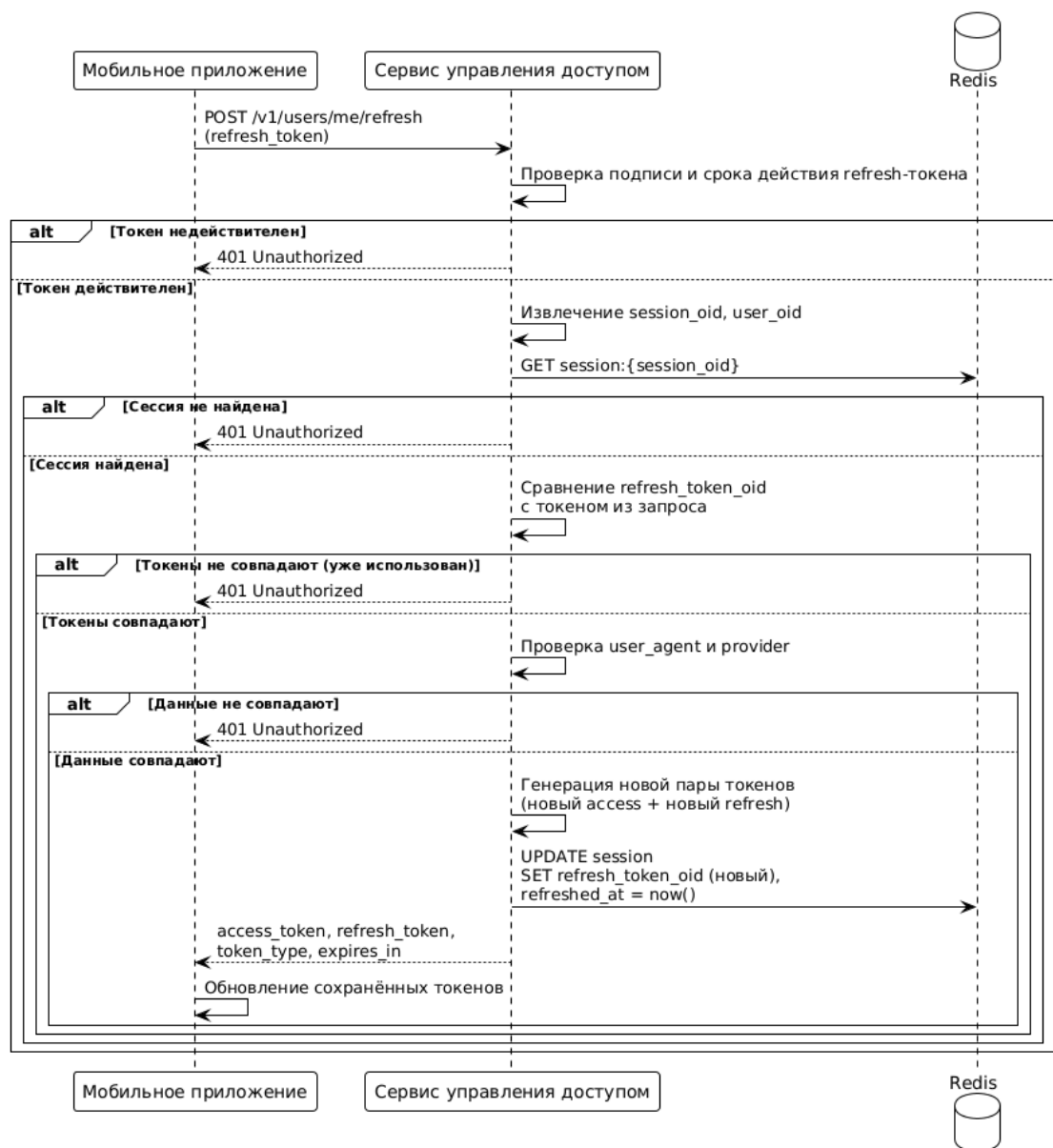


Рисунок А.2 — Диаграмма последовательности обновления токенов

Диаграмма последовательности проверки прав доступа представлена на рисунке А.3.

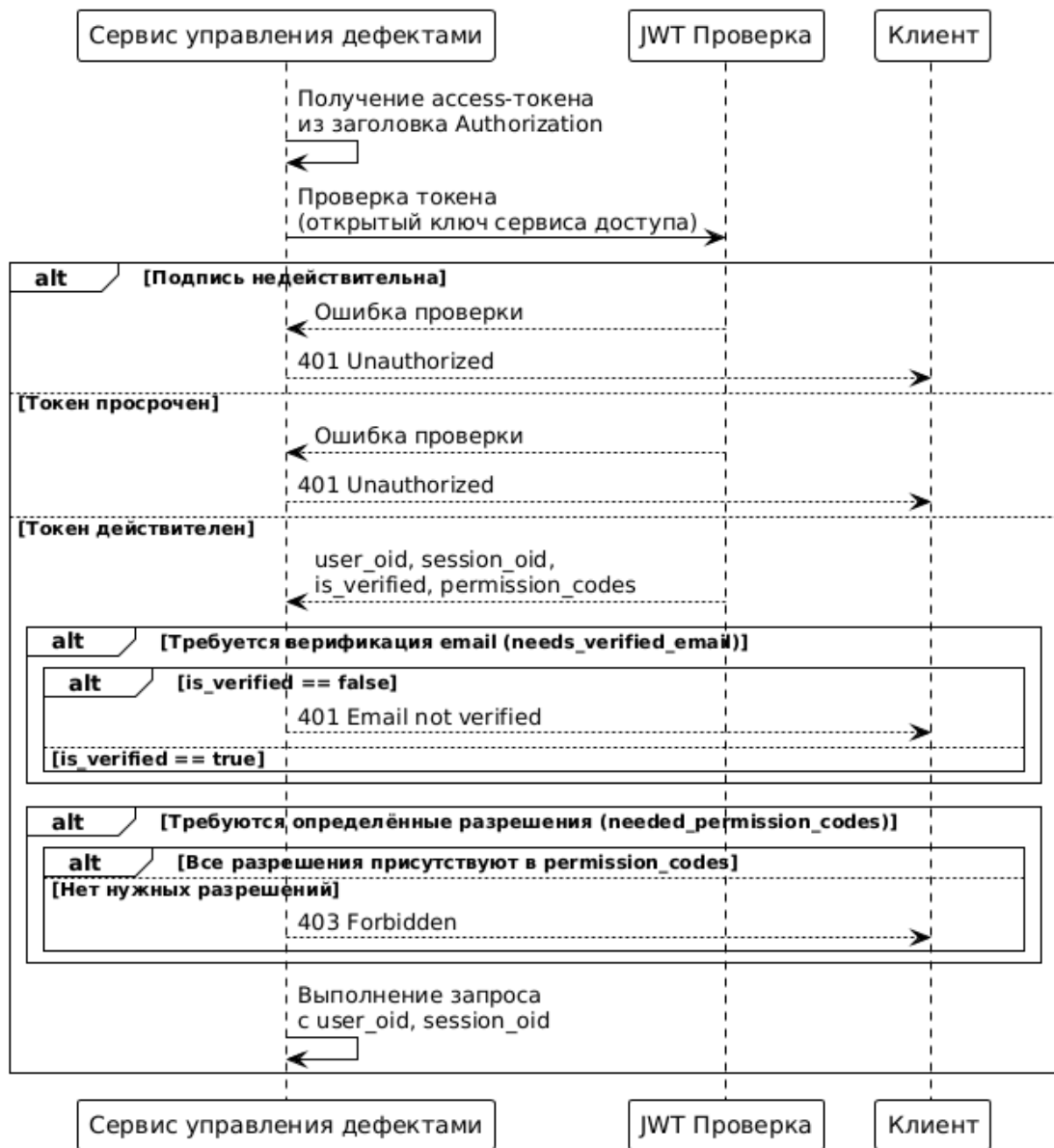


Рисунок А.3 — Диаграмма последовательности проверки прав доступа

ПРИЛОЖЕНИЕ Б

Алгоритмы сервиса управления дефектами

Диаграмма последовательности создания дефекта представлена на рисунке Б.1.

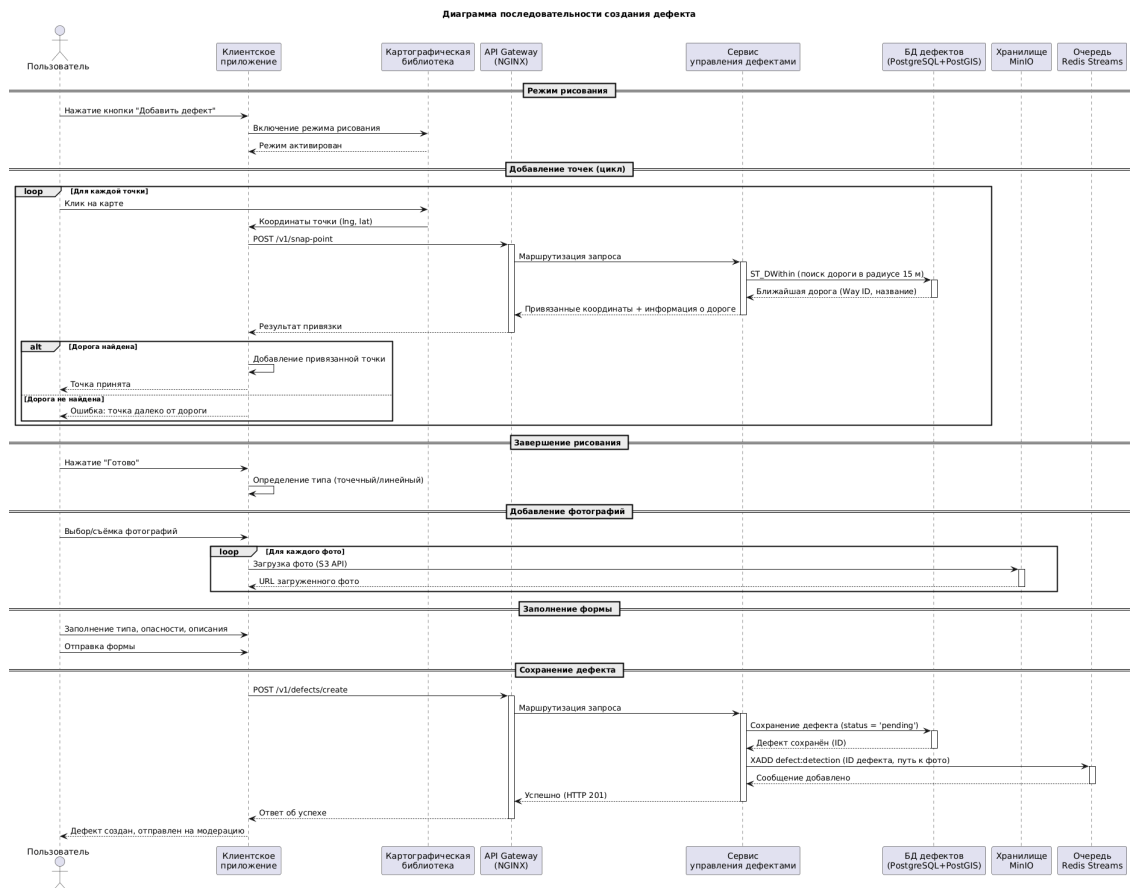


Рисунок Б.1 — Диаграмма последовательности создания дефекта

Диаграмма последовательности поиска дефектов рядом с местоположением представлена на рисунке Б.2.

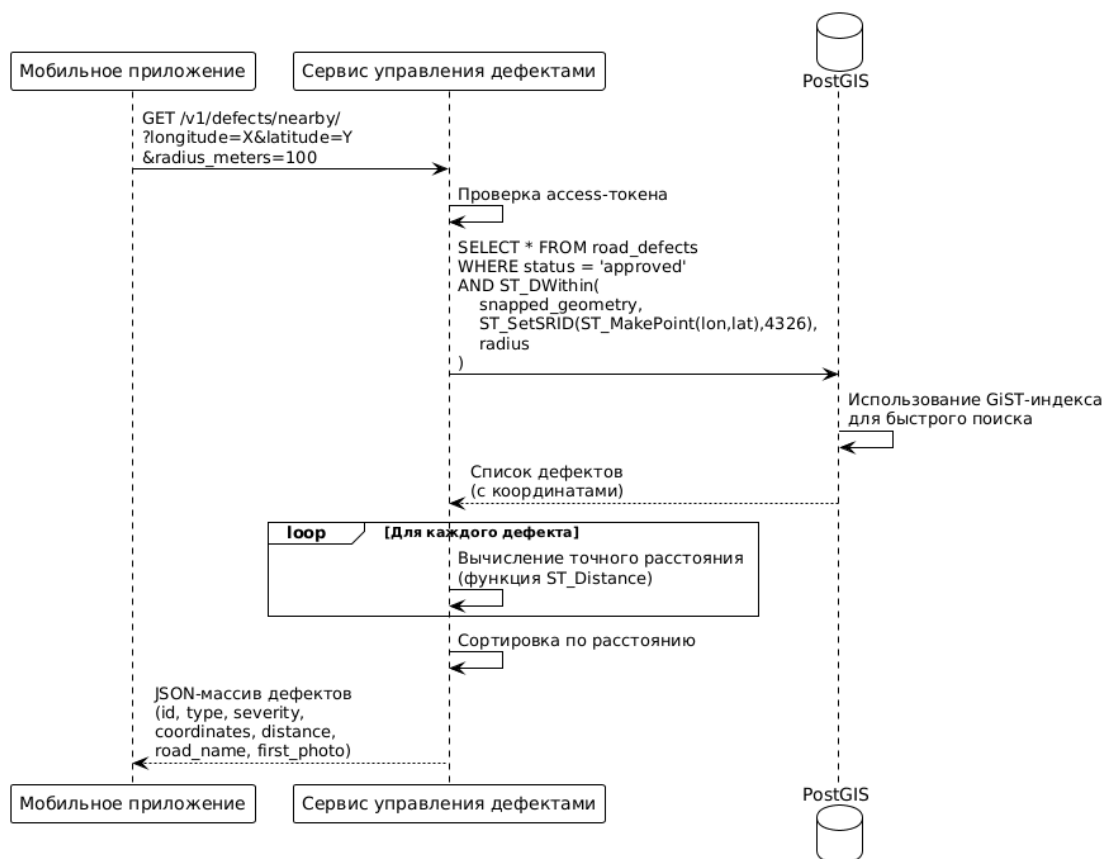


Рисунок Б.2 — Диаграмма последовательности поиска дефектов рядом с местоположением

Диаграмма последовательности поиска дефектов в видимой области карты представлена на рисунке Б.3.

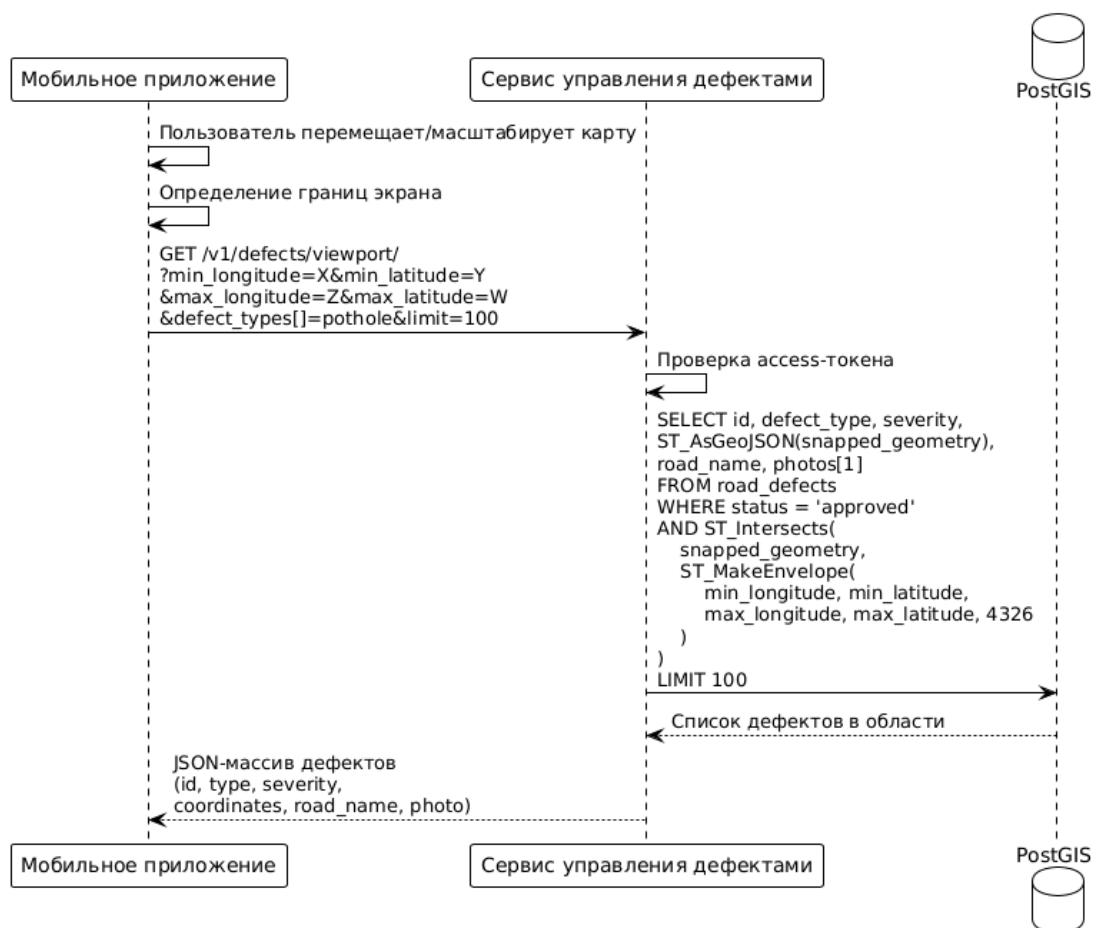


Рисунок Б.3 — Диаграмма последовательности поиска дефектов в видимой области карты

ПРИЛОЖЕНИЕ В

Swagger-документация API

Ниже представлены скриншоты интерактивной документации API (Swagger UI) разработанных микросервисов.

Сервис управления доступом: документация доступна по адресу / api/docs и содержит все эндпоинты для регистрации, аутентификации, управления сессиями, ролями и разрешениями.

Auth Service

0.1.0 OAS 3.1

/openapi.json

Auth Service

Authorize

user

POST /v1/users/register Register User

POST /v1/users/login Login User

POST /v1/users/me/refresh Refresh Token

DELETE /v1/users/me/logout Logout User

DELETE /v1/users/me/logout-all Logout All User

GET /v1/users Get Users

GET /v1/users/me Get Current User

GET /v1/users/me/session Get Current User Session Oids

GET /v1/users/{identifier} Get User By Identifier

POST /v1/users/{user_identifier}/roles/{role_identifier} Assign Role To User

DELETE /v1/users/{user_identifier}/roles/{role_identifier} Revoke Role From User

POST /v1/users/me/verify-by-email Email Verify By Email

GET /v1/users/me/verify Verify User

PATCH /v1/users/{identifier}/activate Activate User

PATCH /v1/users/{identifier}/deactivate Deactivate User

roles

POST /v1/roles/create Role Create

GET /v1/roles Roles Get List

GET /v1/roles/{identifier} Role Get By Identifier

PUT /v1/roles/{identifier} Role Update

DELETE /v1/roles/{identifier} Role Delete

POST /v1/roles/{role_identifier}/permissions/{permission_identifier} Role Assign Permission

DELETE /v1/roles/{role_identifier}/permissions/{permission_identifier} Role Assign Permission

permission

POST /v1/permission/create Role Create

GET /v1/permissions Permissions Get List

GET /v1/permissions/{identifier} Permission Get By Identifier

PUT /v1/permissions/{identifier} Permission Update

DELETE /v1/permissions/{identifier} Permission Delete

Рисунок В.1 — Swagger-документация сервиса управления доступом

Сервис управления дефектами: документация доступна по адресу /api/docs и содержит эндпоинты для создания, модерации, поиска дефектов, а также привязки координат к дорожной сети.

Defects Service 0.1.0 OAS 3.1

/openapi.json

Defects Service

Authorize 

defects ^

POST	/v1/defects/create	Create Defect		▼
GET	/v1/defects/{defect_id}	Get Defect		▼
DELETE	/v1/defects/{defect_id}	Delete Defect		▼
GET	/v1/users/{user_id}/defects	Get Defects By User Id		▼
GET	/v1/defects/nearby/	Get Defects Nearby		▼
GET	/v1/defects	Get Defects		▼
GET	/v1/defects/viewport/	Get Defects In Viewport		▼
GET	/v1/defects/pending/	Get Pending Defects		▼
PATCH	/v1/defects/{defect_id}/moderate	Moderate Defect		▼
PATCH	/v1/defects/{defect_id}/update	Update Defect		▼

roads ^

POST	/v1/snap-point	Snap Point		▼
------	----------------	------------	--	---

Рисунок В.2 — Swagger-документация сервиса управления дефектами

ПРИЛОЖЕНИЕ Г

Интерфейс мобильного приложения

Ниже представлены скриншоты основных экранов и диалоговых окон мобильного приложения.

Экран карты с дефектами: отображает карту с нанесёнными дефектами, маркеры имеют цвет в зависимости от уровня опасности. Реализована кластеризация маркеров на малых масштабах.

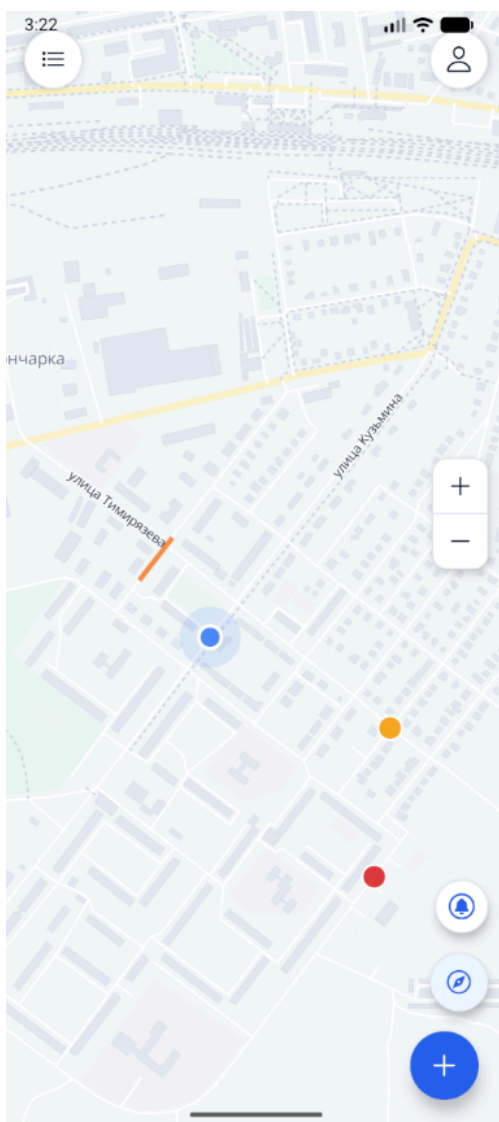


Рисунок Г.1 — Экран карты с дефектами

Попытка добавления дефекта без верификации: система запрашивает подтверждение email перед созданием дефектов.

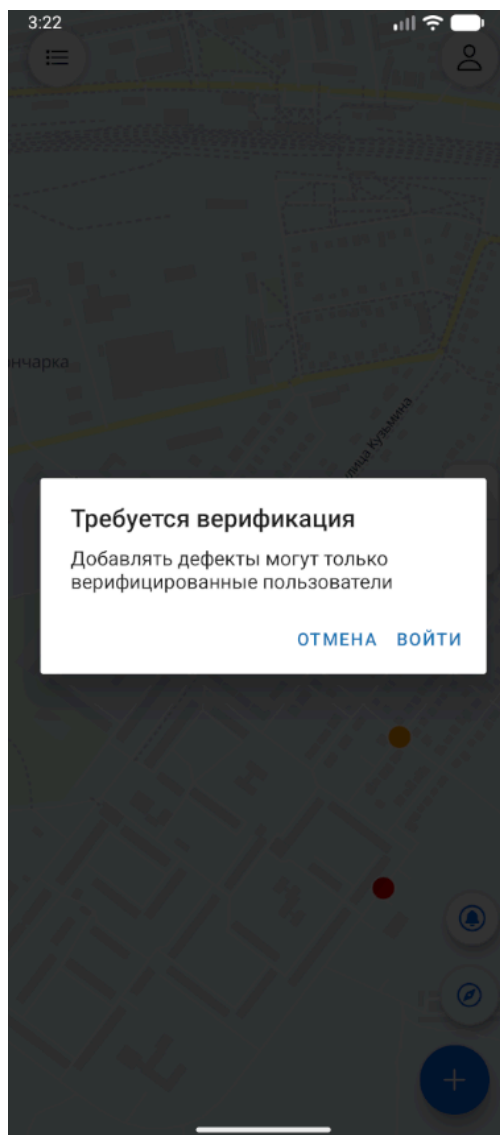


Рисунок Г.2 — Требование верификации email

Окно входа: форма для ввода email и пароля, кнопка входа и перехода к регистрации.

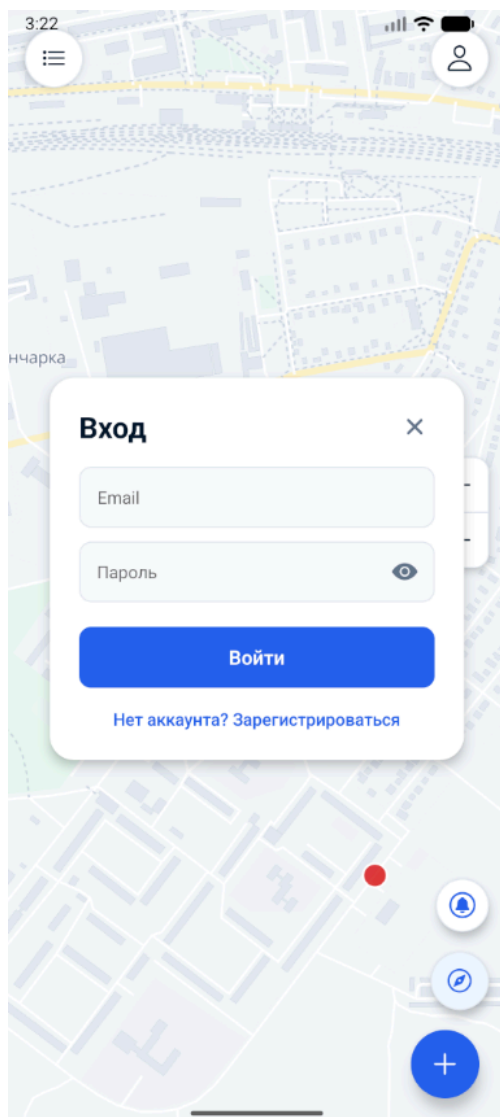


Рисунок Г.3 — Экран входа

Окно регистрации: форма для создания нового аккаунта.

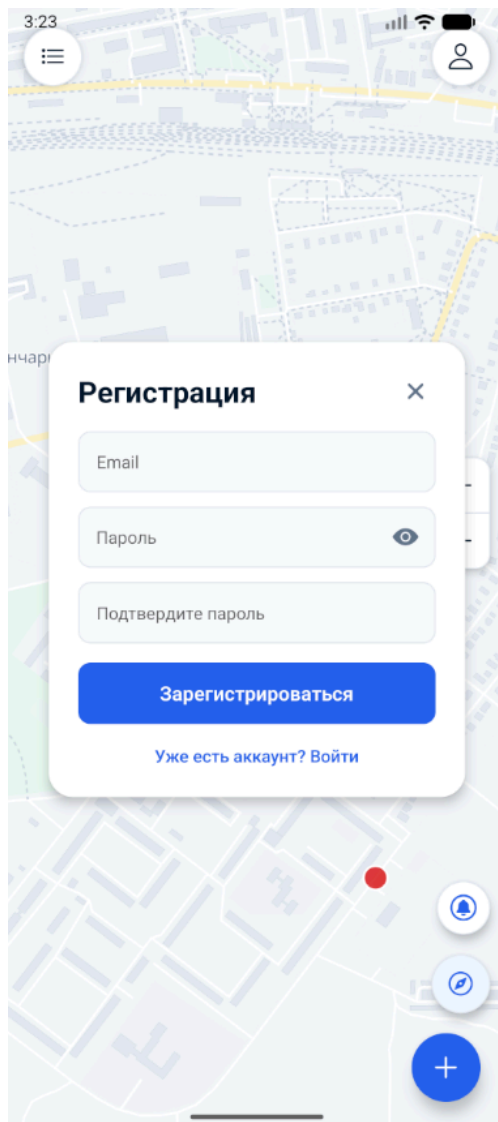


Рисунок Г.4 — Экран регистрации

Профиль пользователя: отображает информацию о пользователе, роли, сессии, кнопки выхода.

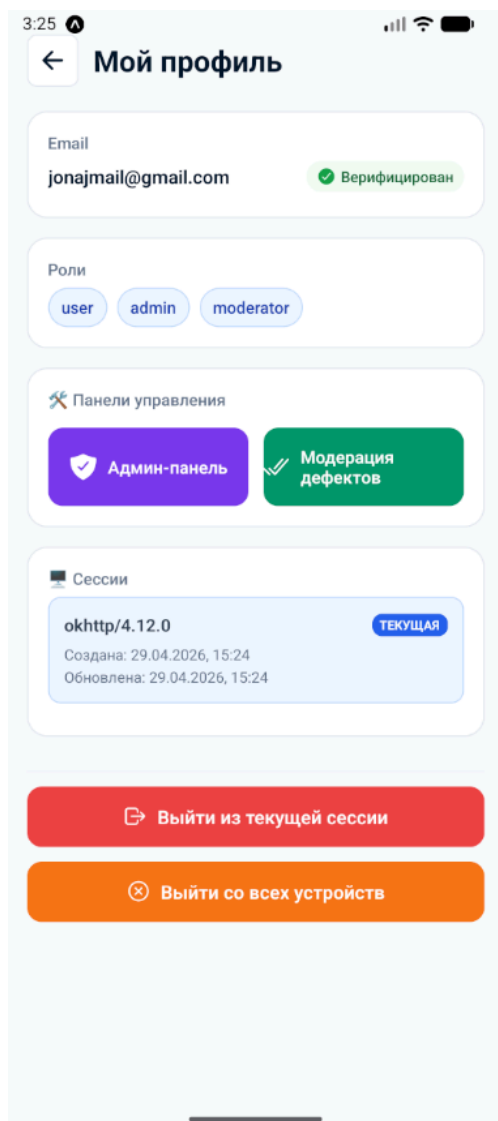


Рисунок Г.5 — Экран профиля

Админ-панель: панель управления для администратора (пользователи, роли, разрешения).

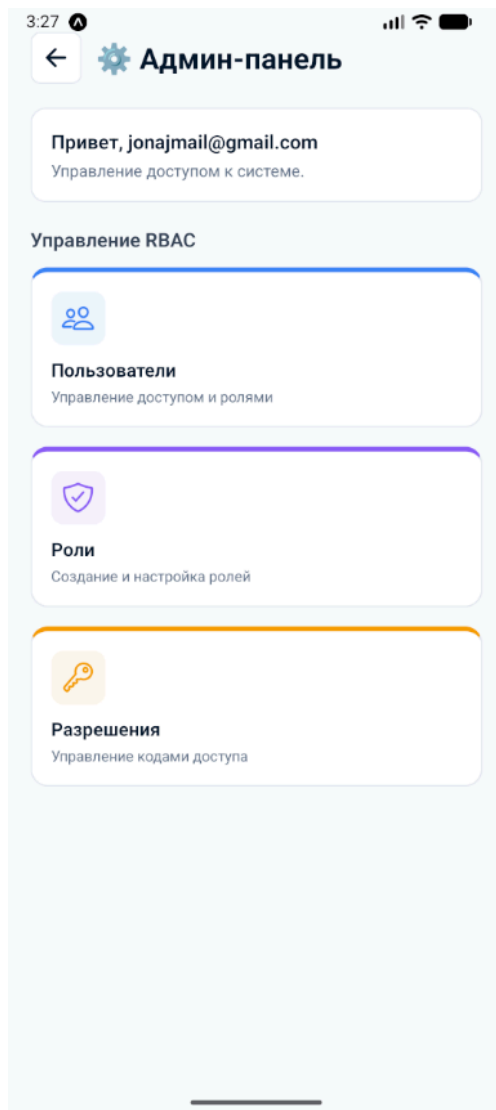


Рисунок Г.6 — Админ-панель

Список пользователей: таблица всех зарегистрированных пользователей с пагинацией.

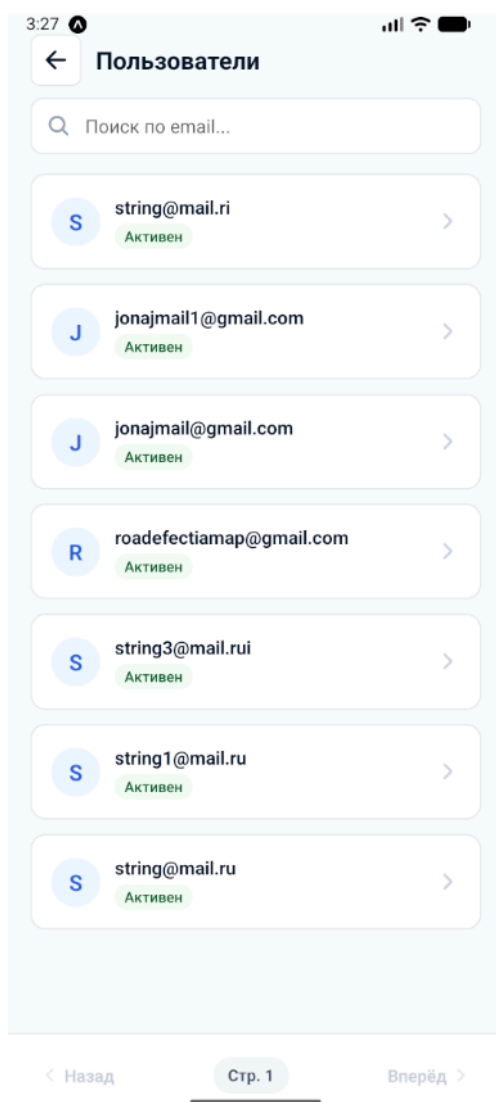


Рисунок Г.7 — Список пользователей

Управление пользователем: детальная информация о пользователе, возможность назначения ролей и активации/деактивации.

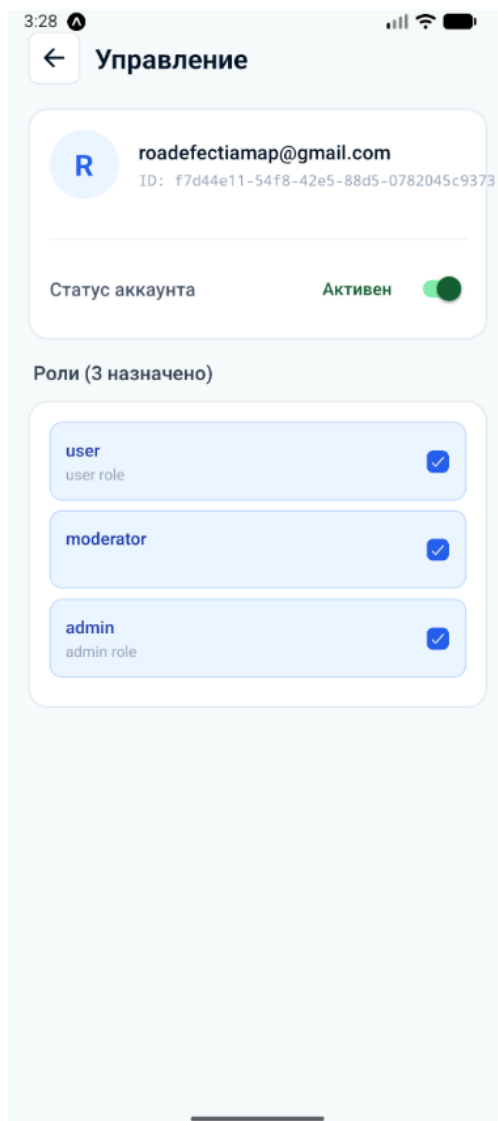


Рисунок Г.8 — Управление пользователем

Управление ролями: список ролей, возможность создания, редактирования и удаления.

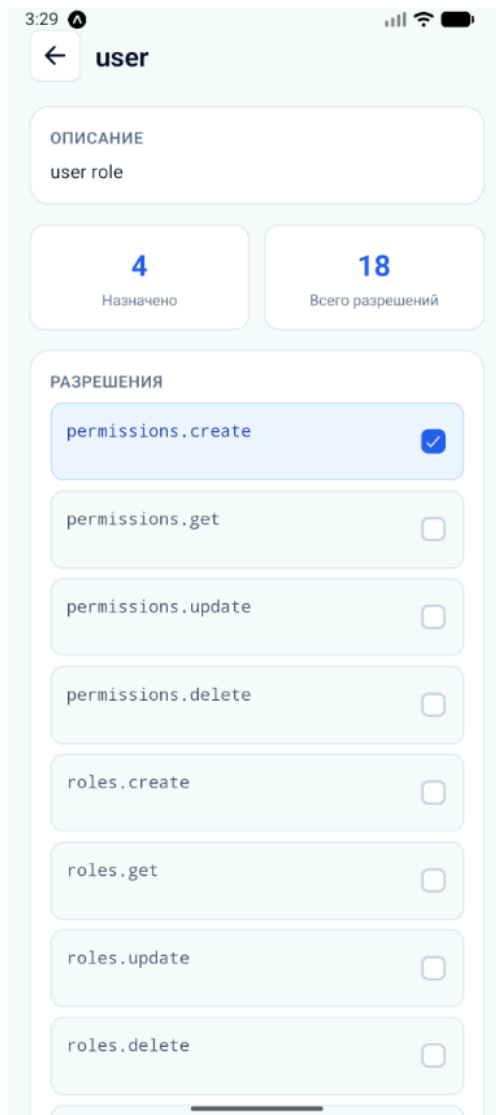


Рисунок Г.9 — Управление ролями

Управление разрешениями: список разрешений, возможность создания и удаления.

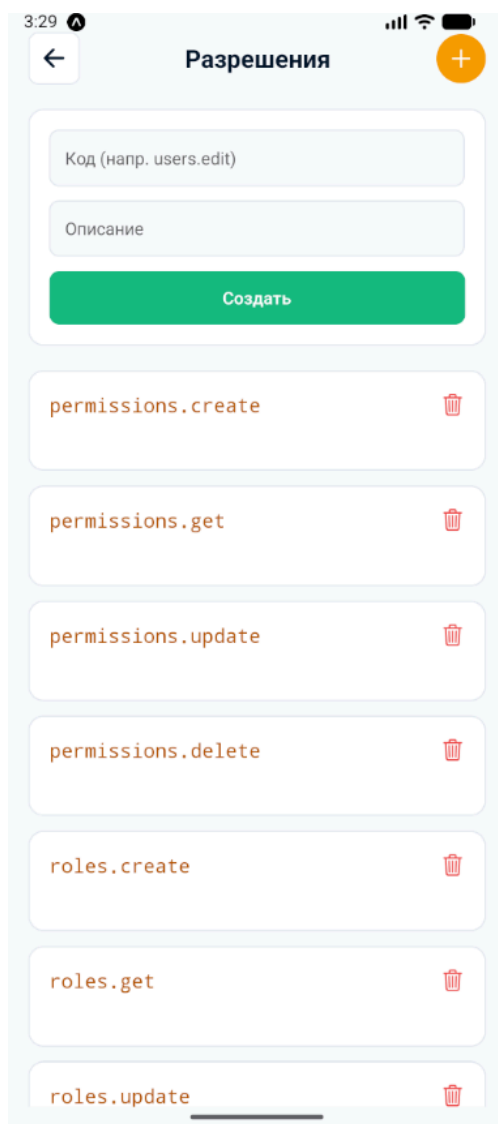


Рисунок Г.10 — Управление разрешениями

Список моих дефектов: отображает дефекты, созданные текущим пользователем, с фильтрацией по статусу.

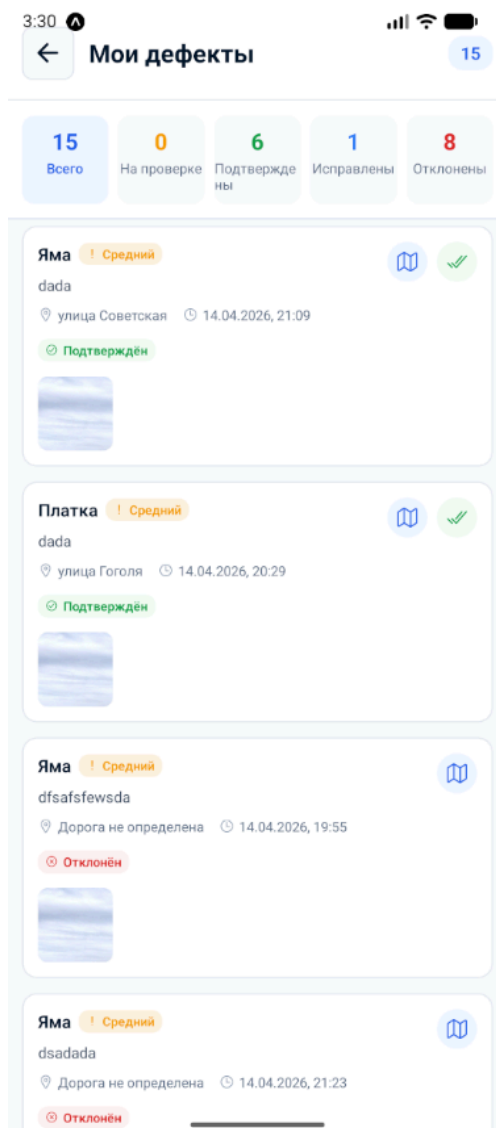


Рисунок Г.11 — Список моих дефектов

Режим рисования на карте: пользователь отмечает точки линейного дефекта на карте.

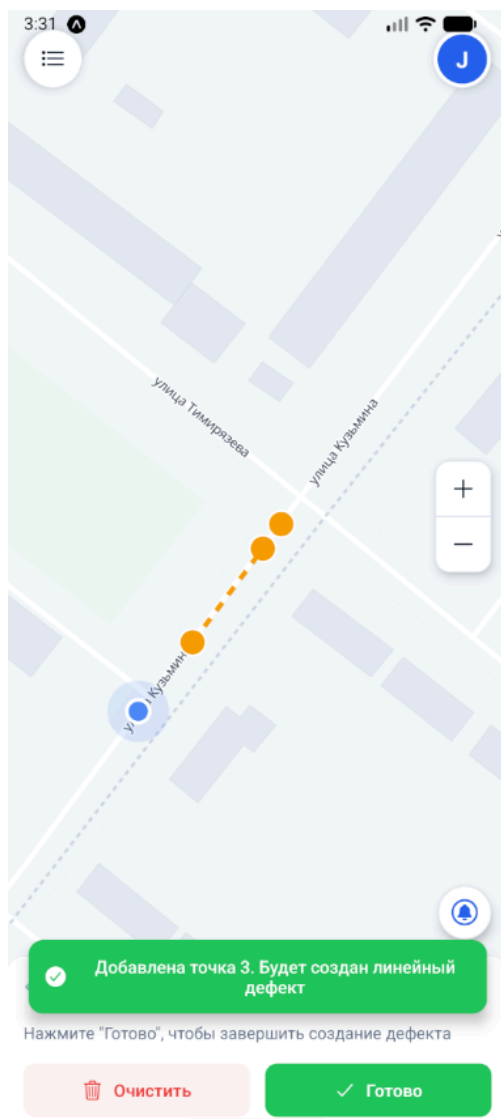


Рисунок Г.12 — Режим рисования дефекта на карте

Форма создания дефекта: выбор типа, уровня опасности, добавление описания и фотографий.

3.36

← Создание дефекта →

Яма

Серьёзность

Низкий Средний Высокий Критический

Описание

Глубокие ямы на дороге, будьте осторожны

39/500

Фотографии

Снять фото Из галереи

Отправить на модерацию

Дефект будет проверен модератором перед публикацией на карте

Рисунок Г.13 — Форма создания дефекта

Панель модератора: дашборд с количеством дефектов на модерации и разделами.

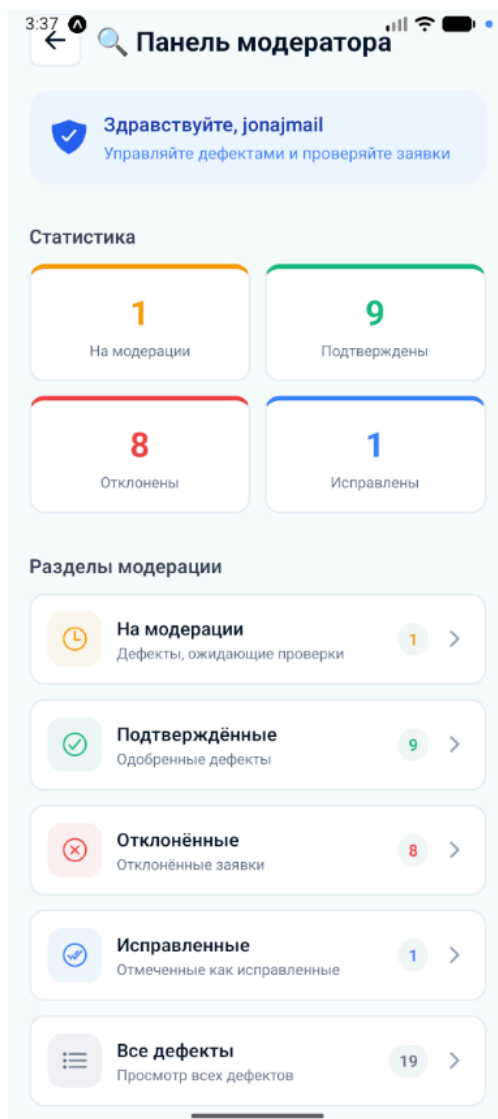


Рисунок Г.14 — Панель модератора

Модерация дефекта: просмотр дефекта с фотографиями, возможность подтвердить или отклонить с указанием причины.

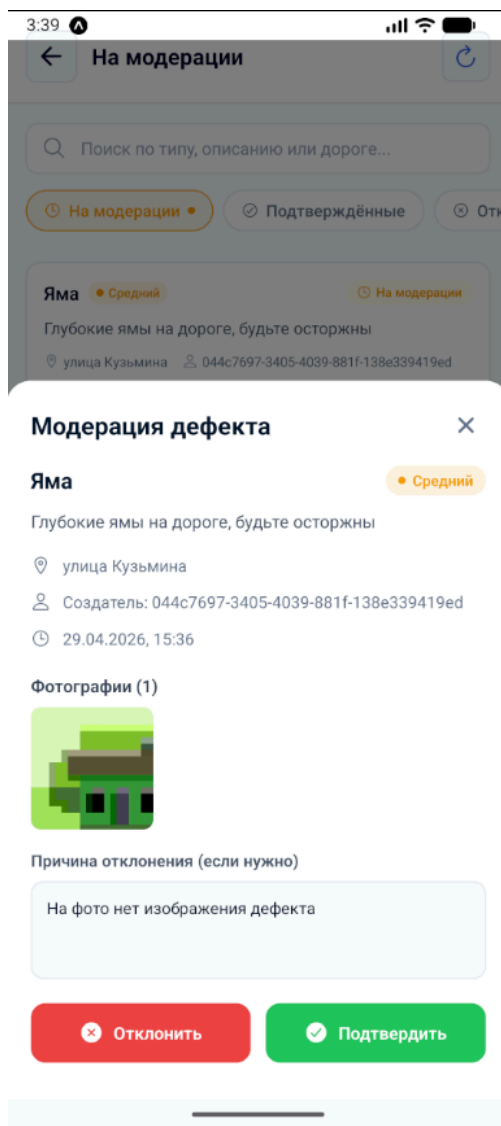


Рисунок Г.15 — Модерация дефекта

Оповещение о дефекте: push-уведомление при приближении к критическому или высокоопасному дефекту.

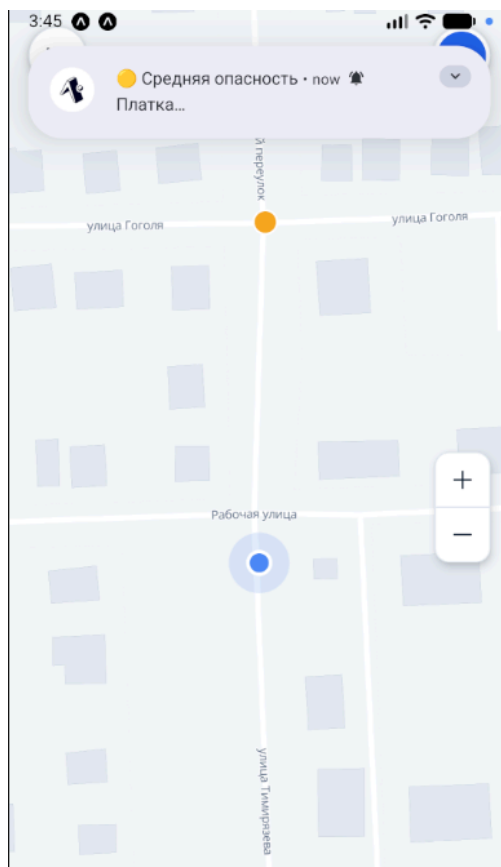


Рисунок Г.16 — Оповещение о приближении к дефекту